EVANGADI STUDY FORUM // ZERO-TO-HERO CURRICULUM

ForumStudy Codebase Master Guide

A comprehensive, line-by-line pedagogical breakdown explaining the **WHY**, **WHEN**, and **HOW** behind every single module, file, function, and architectural decision. Designed specifically for absolute beginners to transition from "Zero Mind to Hero".

69	12,266	10	100%
SOURCE FILES ANALYZED	EXACT LINES EXPLAINED	CHRONOLOGICAL SECTIONS	ZERO-TO-HERO PEDAGOGY

Stack: React 19 + Vite + Express 5 + MySQL + Gemini RAG

Pedagogy: Why, When, & How Framework

Target: Absolute Beginners & Full-Stack Learners

🧠 Chapter 0: The Absolute Beginner's Mental Model

Before reading code, you must understand how all parts fit together. If you memorize code without a mental model, you stay confused. When you understand the model, you become a hero.

🍽️ 1. The Restaurant Analogy (Client vs Server)

Frontend (The Dining Room): What the customer sees—the tables, menus, and buttons (React + HTML/CSS).

Backend API (The Waiters): Takes orders from the dining room, validates requirements, and runs to the kitchen (Express routes & controllers).

Database (The Kitchen Pantry): The permanent storage where all ingredients and customer bills are recorded safely (MySQL).

🔁 2. The 3-Tier Layered Architecture

Route: The URL door (e.g., `POST /api/auth/login`).
Validation: The bouncer inspecting your ticket before entry (express-validator).
Controller: The receptionist receiving the request body and coordinating the response.
Service: The business logic brain performing passwords hashing, SQL execution, and Gemini AI calls.

🧭 3. What are Vector Embeddings?

Computers cannot understand the concept of "debugging a React state bug". They only understand numbers. A **vector embedding** converts text into a list of 1,536 coordinates on a hyper-dimensional globe. Words with similar meanings land near each other. **Cosine similarity** simply calculates the compass angle between those two points!

🚂 4. What is RAG (Retrieval-Augmented Generation)?

Standard AI guesses when asked about your private class notes. **RAG** solves this: 1. We chop your PDF into 900-character paragraphs (chunks).
2. When you ask a question, we find the 5 most relevant paragraphs.
3. We give those paragraphs to Gemini and say: *"Answer strictly using these excerpts, and cite the chunk numbers."* Zero hallucination!

Table of Contents

All 10 Chronological Sections covering the complete 12,266-line system architecture.

- 1. Database & Architecture Blueprint — 4 files
- 2. Backend Server Core & Middleware — 6 files
- 3. Backend Authentication Module — 4 files
- 4. Backend Question & AI Vector / Coach Module — 6 files
- 5. Backend Answer Module — 4 files
- 6. Backend RAG (Document AI) Module — 5 files
- 7. Frontend Core & Configuration — 8 files
- 8. Frontend Contexts & API Services — 6 files
- 9. Frontend Shared Components — 12 files
- 10. Frontend Pages & CSS Modules — 14 files

SECTION 01 // SYSTEM ARCHITECTURE



Relational MySQL schemas, database connection pool, configuration parameters, and safe SQL execution wrappers.

backend/db/schema.sql

135 lines

DATABASE DDL (MYSQL) • DATABASE · SCHEMA

❓ WHY IS IT HERE?

Enforces strict relational integrity, primary keys, cascading deletes, unique constraints, and B-Tree indexes at the database engine level so corrupt or duplicate data can never enter the system.

⏱️ WHEN DOES IT RUN?

Executed during database setup and schema migrations before the backend server begins accepting HTTP requests.

⚙️ HOW DOES IT WORK?

Uses MySQL InnoDB engine with utf8mb4 encoding, defining foreign keys with ON DELETE CASCADE and FULLTEXT indexing for title/content search.

💡 Beginner Concept Spotlight

Primary Keys vs. Foreign Keys: A Primary Key is like a citizen national ID (uniquely identifies a single row). A Foreign Key is a reference pointing to another table's Primary Key (like writing down your parent's ID). ON DELETE CASCADE means if the parent is removed, child records are automatically cleaned up.

⚠️ Junior Developer Trap / Common Gotcha

Never store plain text passwords in the database! Notice the password_hash VARCHAR(255) column designed specifically for 60-character bcrypt strings.

🎯 Technical Interview Pro-Tip

Q: Why is question_hash used for routing instead of question_id?
A: Sequential integer IDs (e.g. /question/1, /question/2) expose business metrics (total questions asked) and allow competitors to scrape the entire database with a simple loop. 16-character cryptographic hashes prevent scraping and enumeration attacks.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

SET FOREIGN_KEY_CHECKS = 0 and 1

Lines 4 & 134

WHY:
Allows dropping and recreating dependent tables in any order without MySQL throwing foreign key constraint errors during migration.

WHEN:
Runs at the very beginning (0) and end (1) of the SQL migration script.

HOW:
Temporarily pauses foreign key validation for the active database session.

Users Table Blueprint

Lines 10-22

WHY:
Maintains user authentication identity. Guarantees email uniqueness and enforces lowercase email consistency.

WHEN:
Inserted on user signup (/api/auth/register); queried on login.

HOW:
CHECK (email = LOWER(email)) ensures case-insensitive matching; password_hash VARCHAR(255) stores one-way cryptographic bcrypt hashes.

Questions Table & FULLTEXT Index

Lines 29-48

WHY:
Holds discussion forum threads. FULLTEXT KEY enables fast keyword searching over millions of words without slow table scans.

WHEN:
Inserted when asking a question; queried for dashboard feeds and search.

HOW:
FULLTEXT KEY ft_questions_search(title, content) enables MATCH(...) AGAINST(...) syntax in MySQL InnoDB.

Question Vectors Table (AI Embeddings)

Lines 54-65

WHY:
Stores Google Gemini semantic vector embeddings linked 1-to-1 with forum questions.

WHEN:
Written immediately after question creation; queried during AI semantic search.

HOW:
Uses MySQL native JSON datatype to store a 1,536-dimensional floating point array representing semantic meaning.

RAG Document & Chunk Tables

Lines 91-133

WHY:
Forms the foundational storage for Retrieval-Augmented Generation (RAG). Slices large PDFs into searchable paragraphs.

WHEN:
Populated when uploading course PDFs in the Knowledge Base.

HOW:
Three-tiered hierarchy: documents (file metadata) → document_chunks (text segments) → document_chunk_vectors (embeddings).

backend/db/config.js

39 lines

DATABASE CONNECTION POOL & QUERY RUNNER (JS) • DATABASE · CONFIG

❓ WHY IS IT HERE?

Opening and closing TCP connections for every HTTP request is slow and crashes the database. A connection pool reuses open connections. safeExecute guarantees all queries use prepared statements.

⏱️ WHEN DOES IT RUN?

Connection pool initializes on server boot; safeExecute executes every time any service queries MySQL.

⚙️ HOW DOES IT WORK?

Uses mysql2/promise createPool(). safeExecute checks that SQL queries are non-empty strings and parameters are arrays/objects before executing.

💡 Beginner Concept Spotlight

What is a Connection Pool? Think of it like a fleet of rental cars at an airport. Instead of buying a new car every time someone arrives, people borrow an idle car, drive to their destination (run query), and return it to the lot (release) for the next traveler.

⚠️ Junior Developer Trap / Common Gotcha

Never use string concatenation or template literals like `SELECT * FROM users WHERE id = ${id}`! Always pass parameters through safeExecute(sql, [id]) with ? placeholders.

🎯 Technical Interview Pro-Tip

Q: How does parameterized execution prevent SQL injection?
A: It sends the SQL query structure and the user data to the database in two separate protocol packets. The database compiles the SQL statement first, meaning user input is strictly treated as literal data and can never alter query logic.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

createPool Configuration

Lines 7-12

WHY:
Maintains an active pool of reusable connections to MySQL with credentials from environment variables.

WHEN:
Created once during Node.js module loading.

HOW:
Reads process.env.DB_HOST, process.env.DB_USER, process.env.DB_PASS, and process.env.DB_NAME.

ensureParams Safety Guard

Lines 14-24

WHY:
Guards against developers forgetting to pass parameter arrays, preventing accidental undefined bindings.

WHEN:
Called inside safeExecute before any database query runs.

HOW:
Checks Array.isArray(params) and typeof params === "object". Throws descriptive error if omitted.

safeExecute Async Function

Lines 29-36

WHY:
The unified, safe highway for all database interactions across all backend services.

WHEN:
Called by controllers and services for every SELECT, INSERT, UPDATE, and DELETE.

HOW:
Executes db.execute(sql, params) using prepared statements. Destructures [result] from the returned tuple and returns it.

backend/package.json

29 lines

BACKEND MANIFEST & DEPENDENCIES (JSON) • CONFIG · BACKEND

❓ WHY IS IT HERE?

Declares required npm packages, configures executable scripts, and enables native ECMAScript Module (ESM) syntax ("type": "module").

⏱️ WHEN DOES IT RUN?

Read by npm during package installation and server startup.

⚙️ HOW DOES IT WORK?

Defines semver constraints (^x.y.z) and scripts: "dev" runs nodemon index.js; "start" runs node index.js.

💡 Beginner Concept Spotlight

CommonJS (require) vs. ES Modules (import): Node.js originally used require(). Modern JavaScript uses "import/export". Setting "type": "module" in package.json unlocks standard modern import syntax.

⚠️ Junior Developer Trap / Common Gotcha

Omitting "type": "module" when using import statements results in the famous "SyntaxError: Cannot use import statement outside a module" error.

🎯 Technical Interview Pro-Tip

Q: What does the caret (^) in "^5.2.1" mean in package.json?
A: The caret allows automatic upgrades to minor and patch releases (e.g. 5.3.0) that do not modify the left-most non-zero digit, protecting your app from breaking major updates (e.g. 6.0.0).

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

NPM Scripts Configuration

Lines 6-10

WHY:
Standardizes CLI developer workflows across all team members and deployment servers.

WHEN:
Executed via npm run dev or npm start.

HOW:
dev: nodemon index.js watches file changes and automatically restarts the server on save.

Production Dependencies

Lines 15-27

WHY:

Express provides REST server; @google/genai powers vector embeddings; bcryptjs hashes passwords; jsonwebtoken signs auth tokens; multer uploads files; pdf-parse extracts text.

WHEN:

Installed into node_modules and bundled in production.

HOW:

Node.js module resolution loads these from disk when imported.

backend/.env.example

45 lines

ENVIRONMENT VARIABLES TEMPLATE (ENV) • CONFIG · ENV

❓ WHY IS IT HERE?

Separates configuration from code (The Twelve-Factor App methodology). Prevents hardcoding secrets into source control.

⏱️ WHEN DOES IT RUN?

Copied to .env and loaded into memory on server boot.

⚙️ HOW DOES IT WORK?

Defines key-value pairs that dotenv parses into process.env.

💡 Beginner Concept Spotlight

Environment Variables: Variables set outside of application code (in the operating system or .env file). They allow running the same codebase in development, staging, and production with different credentials.

⚠️ Junior Developer Trap / Common Gotcha

Never commit your real .env file with actual database passwords or Gemini API keys to GitHub! Add .env to .gitignore and only commit .env.example.

🎯 Technical Interview Pro-Tip

Q: What is the purpose of RAG_CHUNK_OVERLAP=120?
A: When splitting large text documents into chunks (e.g. 900 characters), key sentences often get cut directly across chunk boundaries. Overlapping consecutive chunks by 120 characters preserves context so search queries find complete thoughts.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Server, Database, and JWT Keys

Lines 1-13

WHY:
Defines port 3777, local MySQL credentials, and signing secrets for user authentication tokens.

WHEN:
Loaded once when backend starts up.

HOW:
dotenv reads .env and assigns process.env.PORT, process.env.DB_USER, etc.

Google Gemini AI Configuration

Lines 15-27

WHY:
Sets API credentials for Google Gemini models and configures the 0.75 cosine similarity threshold for question recommendations.

WHEN:
Used during vector generation and question similarity matching.

HOW:
vector.service.js reads GEMINI_EMBEDDING_MODEL and RECOMMEND_THRESHOLD.

RAG Ingestion & Search Hyperparameters

Lines 29-45

WHY:
Controls sliding window character lengths (900 chars), overlap (120 chars), file upload directory (uploads/rag), and search threshold (0.45).

WHEN:
Used by rag.service.js during PDF text extraction, chunking, and querying.

HOW:
Allows tweaking RAG chunk granularity and safety caps without touching source code.

SECTION 02 // SYSTEM ARCHITECTURE



Server bootstrap, central routing junction, JWT authentication guards, input validation, and global error handling.

backend/index.js

53 lines

SERVER ENTRY POINT (JS) • BACKEND · SERVER

❓ WHY IS IT HERE?

The main orchestrator. Sets up the HTTP server, mounts middleware in order, validates database connectivity on startup, and opens port 3777.

⏱️ WHEN DOES IT RUN?

The very first file executed when launching the backend.

⚙️ HOW DOES IT WORK?

Calls express(), registers middleware via app.use(), verifies MySQL pool with db.getConnection(), and calls app.listen().

💡 Beginner Concept Spotlight

What is Middleware? Middleware functions are like airport security checkpoints between the passenger (incoming HTTP request) and the airplane seat (controller). Each checkpoint can inspect, alter, or reject the request before it reaches the destination.

⚠️ Junior Developer Trap / Common Gotcha

Order of middleware matters! app.use(errorHandler) must be placed AFTER app.use('/api', mainRouter), otherwise errors thrown in routes will never reach it.

🎯 Technical Interview Pro-Tip

Q: Why is connection.release() called inside startServer after testing the database?
A: If you acquire a connection from the pool with db.getConnection() and forget to call connection.release(), that connection remains locked and unavailable to other requests, eventually causing the entire pool to freeze.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Express App & Global Middleware Pipeline

Lines 8-16

WHY:

Enables CORS so the React frontend on port 5173 can call port 3777; parses JSON request bodies into req.body.

WHEN:

Executes sequentially on every single incoming HTTP request.

HOW:

app.use(cors()), app.use(express.json()), and app.use(express.urlencoded({ extended: true })).

Healthcheck, Router Mounting & Error Interceptor

Lines 18-25

WHY:
Provides /health endpoint for server monitoring and attaches mainRouter at /api. Mounts global error handler last.

WHEN:
Matches /health or routes under /api, forwarding any thrown errors to errorHandler.

HOW:
app.use('/api', mainRouter);
app.use(errorHandler);

startServer Function with Connection Test

Lines 28-52

WHY:
Guarantees the server does not boot in a broken state if MySQL is unreachable.

WHEN:
Called once when node index.js runs.

HOW:
Awaits db.getConnection(), logs success, releases connection, and calls app.listen(port). Exits with process.exit(1) on failure.

backend/src/api/routes.js

19 lines

CENTRAL API ROUTER (JS) • BACKEND · ROUTING

❓ WHY IS IT HERE?

Keeps index.js clean by organizing endpoints by domain responsibility.

⏱️ WHEN DOES IT RUN?

Loaded on startup to construct the URL dispatch table.

⚙️ HOW DOES IT WORK?

Uses express.Router() to mount authRoutes, questionRoutes, answerRoutes, and ragRoutes.

💡 Beginner Concept Spotlight

Modular Routing: Breaking down a massive web app into distinct specialized sub-apps (Auth router handles login/register; Question router handles posts; RAG router handles PDFs).

⚠️ Junior Developer Trap / Common Gotcha

Always ensure your router subpaths do not repeat their mount prefixes (e.g., in auth.routes.js use router.post('/login'), NOT router.post('/api/auth/login')).

🎯 Technical Interview Pro-Tip

Q: What happens under the hood when a client sends a request to /api/questions/search?
A: Express matches /api in index.js, delegates to mainRouter, matches /questions, delegates to questionRoutes, and finally executes the GET /search controller handler.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Sub-Router Dispatching

Lines 1-19

WHY:
Organizes API URLs into clear RESTful resource categories.

WHEN:
Routes incoming requests based on URL path.

HOW:
mainRouter.use('/auth', authRoutes);
mainRouter.use('/questions', questionRoutes); etc.

backend/src/utils/errors/index.js

42 lines

CUSTOM API ERROR CLASSES (JS) • BACKEND · ERRORS

❓ WHY IS IT HERE?

Standard JavaScript errors only have a message and stack trace. Custom error classes attach the correct HTTP status code (400, 401, 404, 503) directly to the error object.

⏱️ WHEN DOES IT RUN?

Instantiated and thrown anywhere in services or validations when a problem is detected.

⚙️ HOW DOES IT WORK?

Subclasses built-in Error using class inheritance (extends Error) and calls super(message).

💡 Beginner Concept Spotlight

Class Inheritance: Child classes (like BadRequestError) inherit all properties and methods of their parent class (CustomAPIError and Error), allowing them to add specialized properties like this.statusCode = 400.

⚠️ Junior Developer Trap / Common Gotcha

Remember that throwing an error immediately halts the execution of the current function and jumps up the call stack to the nearest try/catch or error middleware.

🎯 Technical Interview Pro-Tip

Q: Why is throwing custom error classes preferred over returning { error: true } from services?
A: Returning error objects requires every intermediate function to check if the result is an error. Throwing errors leverages language-level exception bubbling, creating cleaner code with centralized error handling.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

CustomAPIError Base Class

Lines 3-7

WHY:
Root parent error class for all application-specific errors.

WHEN:
Extended by specific HTTP status error classes.

HOW:
Calls super(message) to preserve V8 stack traces.

HTTP Status Error Classes

Lines 10-39

WHY:

Predefined status codes: BadRequestError (400), NotFoundError (404), UnauthenticatedError (401), ServiceUnavailableError (503).

WHEN:

Thrown when validations fail, items are missing, tokens are invalid, or external APIs fail.

HOW:

Sets this.statusCode using StatusCodes constants from http-status-codes.

backend/src/middleware/authentication.js

32 lines

JWT AUTHENTICATION GUARD (JS) • BACKEND · MIDDLEWARE

❓ WHY IS IT HERE?

Protects private endpoints. Ensures only logged-in users with a valid cryptographic signature can post questions, submit answers, or upload PDFs.

⏱️ WHEN DOES IT RUN?

Executes before controller handlers on all protected routes.

⚙️ HOW DOES IT WORK?

Extracts Authorization: Bearer <token>, verifies it using jwt.verify() with JWT_SECRET, and sets req.user = { id, firstName, lastName }.

💡 Beginner Concept Spotlight

JSON Web Token (JWT): Think of a JWT like a theme park wristband. When you show your ID at the gate (login), you get a tamper-proof wristband. For every ride (API route), you simply flash the wristband. The operator verifies the signature without needing to look up your file in the office.

⚠️ Junior Developer Trap / Common Gotcha

If a token has expired or is altered by 1 character, jwt.verify throws an exception. We catch it and throw UnauthenticatedError('Authentication invalid').

🎯 Technical Interview Pro-Tip

Q: What is the difference between Authentication (401) and Authorization (403)?
A: Authentication verifies WHO you are (e.g. valid login token). Authorization verifies WHAT you are allowed to do (e.g. admin permissions, only author can delete).

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

JWT Secret Guard

Lines 5-9

WHY:
Prevents the server from running insecurely without a cryptographic signing secret.

WHEN:
Checked on module startup.

HOW:
Throws an Error immediately if process.env.JWT_SECRET is undefined.

Bearer Header Extraction

Lines 11-18

WHY:
Checks for standard HTTP Authorization header format.

WHEN:
Evaluated for every incoming request.

HOW:
Checks startsWith('Bearer ') and splits the string to extract the raw token.

jwt.verify and req.user Population

Lines 20-31

WHY:
Validates token authenticity and attaches user identity to the request object for downstream controllers.

WHEN:
Runs on valid tokens; catches expired or forged tokens.

HOW:
jwt.verify(token, JWT_SECRET). Assigns req.user = { id, firstName, lastName } and calls next().

backend/src/middleware/validation-handler.js

12 lines

VALIDATION ERROR INTERCEPTOR (JS) • BACKEND · MIDDLEWARE

❓ WHY IS IT HERE?

Collects all validation errors generated by express-validator rules and returns a readable HTTP 400 response.

⏱️ WHEN DOES IT RUN?

Runs immediately after validation rules and before controller execution.

⚙️ HOW DOES IT WORK?

Calls validationResult(req). If errors exist, extracts error messages and throws BadRequestError.

💡 Beginner Concept Spotlight

Fail Fast Principle: Stop bad requests as early as possible. If an email is malformed or password is too short, reject it before running database queries or AI operations.

⚠️ Junior Developer Trap / Common Gotcha

Always place validationErrorHandler as the final item in your validation array, otherwise validation checks will execute but errors will never be reported!

🎯 Technical Interview Pro-Tip

Q: Why should validation logic be separated from controller logic?
A: Separation of concerns. Controllers should only focus on handling business workflows, not verifying whether string lengths meet minimum requirements.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

validationErrorHandler Middleware

Lines 4-11

WHY:
Bridges express-validator rules with our custom error handling system.

WHEN:
Executes after input validation checks.

HOW:
validationResult(req). If !errors.isEmpty(), maps err.msg into a single string and throws BadRequestError.

backend/src/middleware/error-handler.js

29 lines

GLOBAL ERROR HANDLER (JS) • BACKEND · MIDDLEWARE

❓ WHY IS IT HERE?

Centralized error response formatter. Prevents unhandled exceptions from crashing the server and hides sensitive internal stack traces from production clients.

⏱️ WHEN DOES IT RUN?

Called automatically whenever next(error) is called or an exception is thrown in route processing.

⚙️ HOW DOES IT WORK?

Express identifies it by its 4 arguments: (err, req, res, next). Checks for MySQL ER_DUP_ENTRY and returns res.status().json({ msg }).

💡 Beginner Concept Spotlight

Express 4-Argument Signature: Standard Express middleware takes 3 arguments (req, res, next). An error-handling middleware MUST take 4 arguments: (err, req, res, next). Express uses function arity to distinguish between regular and error middleware.

⚠️ Junior Developer Trap / Common Gotcha

If you fail to handle database errors like ER_DUP_ENTRY gracefully, users attempting to register with an existing email will receive an intimidating 500 Server Error instead of a helpful "User already exists" message.

🎯 Technical Interview Pro-Tip

Q: Why should production error handlers never send err.stack to clients?
A: Stack traces reveal server file paths, library versions, and database schemas, giving malicious attackers valuable reconnaissance data for exploiting vulnerabilities.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Error Normalization

Lines 3-8

WHY:
Ensures every error has an HTTP status code and a human-readable message.

WHEN:
Runs on every caught error.

HOW:
customError.statusCode defaults to err.statusCode or 500; msg defaults to err.message or fallback string.

MySQL Duplicate Key Interception

Lines 9-15

WHY:
Translates raw database duplicate key errors (ER_DUP_ENTRY) into friendly HTTP 400 Bad Request messages.

WHEN:
Triggered when a unique constraint in MySQL is violated (e.g. registering with an existing email).

HOW:
Checks err?.code === "ER_DUP_ENTRY", sets statusCode = 400, and returns res.status(400).json({ msg }).

SECTION 03 // SYSTEM ARCHITECTURE



User registration with bcrypt password hashing, login verification, and JWT issuance.

backend/src/api/auth/routes/auth.routes.js

29 lines

AUTH ROUTE ENDPOINTS (JS) • AUTH · ROUTE

❓ WHY IS IT HERE?

Defines the public HTTP entry points for user onboarding and session creation.

⏱️ WHEN DOES IT RUN?

Receives user registration and login requests from the frontend auth form.

⚙️ HOW DOES IT WORK?

Connects express.Router() to validation middleware and controller functions.

💡 Beginner Concept Spotlight

HTTP POST Method: POST is used when submitting sensitive data (passwords) or creating new database records. Unlike GET, POST data is sent in the encrypted request body, not exposed in the browser URL bar.

⚠️ Junior Developer Trap / Common Gotcha

Always ensure login and register routes are public! Do not attach authenticateUser middleware to these routes, or users could never sign in!

🎯 Technical Interview Pro-Tip

Q: Why should login and registration be POST requests instead of GET?
A: GET requests append parameters to the URL query string, which gets logged in browser history, proxy server logs, and web server access logs. POST sends credentials in the HTTP body, keeping passwords private.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Register and Login Route Pipeline

Lines 19 & 26

WHY:
Wires up URL endpoints with their respective validation chains and controllers.

WHEN:
Triggered when client POSTs to /register or /login.

HOW:
router.post('/register', registerValidation, registerController);
router.post('/login', loginValidation, loginController);

backend/src/api/auth/validations/auth.validation.js

45 lines

AUTH VALIDATION RULES (JS) • AUTH · VALIDATION

❓ WHY IS IT HERE?

Protects the database from garbage data, prevents SQL bloat, and enforces minimum security requirements on passwords.

⏱️ WHEN DOES IT RUN?

Executes before auth controllers run.

⚙️ HOW DOES IT WORK?

Uses express-validator body() chaining rules: .notEmpty(), .isEmail(), .normalizeEmail(), .isLength({ min: 6 }).

💡 Beginner Concept Spotlight

Sanitization vs. Validation: Validation checks if data matches rules (e.g. is this an email?). Sanitization cleans data (e.g. normalizeEmail() trims whitespace and converts John@Doe.COM to john@doe.com).

⚠️ Junior Developer Trap / Common Gotcha

Failing to normalize emails means User@test.com and user@test.com could be created as separate accounts, causing user confusion and login issues.

🎯 Technical Interview Pro-Tip

Q: Why enforce a minimum password length (e.g. 6+ characters) on the backend?
A: Short passwords can be cracked in seconds via brute-force dictionary attacks. Enforcing minimum length increases the cryptographic entropy required to guess passwords.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

registerValidation Schema

Lines 4–32

WHY: Ensures firstName and lastName are strings >= 3 chars, email is valid and normalized, and password >= 6 chars.

WHEN: Runs on POST /api/auth/register.

HOW: Chained express-validator rules concluding with validationErrorHandler.

loginValidation Schema

Lines 34-44

WHY:

Ensures login requests contain non-empty, normalized email and password.

WHEN:

Runs on POST /api/auth/login.

HOW:

```
body('email').isEmail().normalizeEmail();
body('password').notEmpty();
```

backend/src/api/auth/controller/auth.controller.js

57 lines

AUTH REQUEST CONTROLLERS (JS) • AUTH · CONTROLLER

❓ WHY IS IT HERE?

Controllers act as the HTTP layer orchestrator: they unpack req.body, call services to perform business logic, and send appropriate HTTP status codes (201 Created, 200 OK).

⏱️ WHEN DOES IT RUN?

Executed after validation passes.

⚙️ HOW DOES IT WORK?

Extracts { firstName, lastName, email, password } from req.body, awaits authService, and returns JSON response.

💡 Beginner Concept Spotlight

HTTP 201 Created vs. 200 OK: 201 Created is the standard REST status code returned when a brand new resource (a new user account) is successfully written to the database. 200 OK is returned for successful retrieval or actions (like logging in).

⚠️ Junior Developer Trap / Common Gotcha

Always wrap async controller code in try/catch and pass errors to next(error). If you forget catch(error) { next(error); }, an unhandled promise rejection can hang the client request indefinitely!

🎯 Technical Interview Pro-Tip

Q: Why should controllers never contain SQL queries directly?
A: Layered architecture (Controllers vs. Services). If business rules or database engines change, you only update the service layer without touching the HTTP controllers.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

registerController

Lines 12-31

WHY:
Orchestrates user creation and sends HTTP 201 with new user data.

WHEN:
POST /api/auth/register.

HOW:
Awaits registerService(req.body) and responds with StatusCodes.CREATED (201).

loginController

Lines 41-56

WHY:
Orchestrates user verification and sends HTTP 200 with JWT token.

WHEN:
POST /api/auth/login.

HOW:
Awaits loginService(req.body) and responds with StatusCodes.OK (200), user object, and token.

backend/src/api/auth/service/auth.service.js

125 lines

AUTH BUSINESS LOGIC (JS) • AUTH · SERVICE

❓ WHY IS IT HERE?

The core security engine. Handles user uniqueness checks, password hashing with cryptographic salt, database insertion, and token generation.

⏱️ WHEN DOES IT RUN?

Called by auth.controller.js during signup or signin.

⚙️ HOW DOES IT WORK?

Uses bcrypt.genSalt(10), bcrypt.hash(), bcrypt.compare(), and jwt.sign() with JWT_SECRET.

💡 Beginner Concept Spotlight

What is Salt in Cryptography? A salt is a random string added to the password before hashing. Even if two users choose the identical password "password123", their salt will be different, resulting in completely different hashes. This renders pre-computed hacker "rainbow tables" useless!

⚠️ Junior Developer Trap / Common Gotcha

Never return the password_hash in the response object sent to the frontend! Notice lines 73-78 and 115-121 return only { id, firstName, lastName, email } without password_hash.

🎯 Technical Interview Pro-Tip

Q: How does bcrypt.compare() know the salt if the salt isn't stored in a separate database column?
A: The bcrypt hash string embeds the algorithm version, cost factor (10), 22-character salt, and cipher hash together separated by dollar signs (e.g. $2a$10$N9qo8uLOickgx2ZMRZoMye...). bcrypt.compare() extracts the salt directly from the stored hash.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

checkUserExists Function

Lines 24-29

WHY:
Queries MySQL to verify whether an email is already registered.

WHEN:
Runs before attempting to register a new user.

HOW:
Executes SELECT user_id FROM users WHERE email = ? and returns rows.length > 0.

registerService Implementation

Lines 41-79

WHY:
Checks for duplicates, generates random salt, hashes password, inserts user, and returns user ID.

WHEN:
Called during new user signup.

HOW:
bcrypt.genSalt(10) → bcrypt.hash(password, salt) → safeExecute(INSERT INTO users...) → returns { id, firstName, lastName, email }.

loginService Implementation

Lines 90-124

WHY:
Validates user credentials, compares password hash, and issues a signed JWT token.

WHEN:
Called during user login.

HOW:
SELECT user by email → bcrypt.compare(password, user.password_hash) → jwt.sign(payload, JWT_SECRET, { expiresIn }) → returns user & token.

SECTION 04 // SYSTEM ARCHITECTURE



Question posting, semantic search, AI draft coaching with Gemini, and vector cosine similarity matching.

backend/src/api/question/routes/question.routes.js

75 lines

QUESTION API ROUTES (JS) • QUESTION · ROUTE

❓ WHY IS IT HERE?

Defines RESTful routes for the entire question lifecycle and AI enhancements.

⏱️ WHEN DOES IT RUN?

Mounted under /api/questions on server startup.

⚙️ HOW DOES IT WORK?

Express Router connects endpoints to authentication guard, validators, and controllers.

💡 Beginner Concept Spotlight

Static Routes vs. Dynamic Param Routes: Static routes like /search and /draft-coach MUST be defined BEFORE dynamic parameter routes like /:questionHash. If /:questionHash came first, Express would think "search" was a question hash ID!

⚠️ Junior Developer Trap / Common Gotcha

Placing router.get('/:questionHash') before router.get('/search') is a common bug where navigating to /api/questions/search triggers the single-question handler with questionHash = "search"!

🎯 Technical Interview Pro-Tip

Q: How do route orders in Express prevent routing conflicts?
A: Express evaluates routes top-to-bottom and stops at the first matching pattern. Specific literal routes must always precede generic parameterized routes (like /:id).

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Specific Routes (POST /, GET /, GET /search, POST /draft-coach)

Lines 25-51

WHY:
Handles question creation, listing, semantic AI search, and draft coaching.

WHEN:
Called when loading dashboard, typing search queries, or requesting AI draft tips.

HOW:
Guarded with authenticateUser, validated, and passed to controllers.

Dynamic Parameter Routes (/:questionHash, /:questionHash/similar)

Lines 54-73

WHY:
Fetches individual threads, related questions, and checks answer fit.

WHEN:
Called when viewing a question or testing an answer draft against a question.

HOW:
Extracts :questionHash from req.params.

backend/src/api/question/validations/question.validation.js

121 lines

QUESTION VALIDATION SCHEMAS (JS) • QUESTION · VALIDATION

❓ WHY IS IT HERE?

Ensures question submissions meet minimum quality criteria and validates hash formats with regular expressions.

⏱️ WHEN DOES IT RUN?

Runs before question controller execution.

⚙️ HOW DOES IT WORK?

Uses HASH_REGEX = /^[a-f0-9]{16}$/ and express-validator body, param, query inspectors.

💡 Beginner Concept Spotlight

Regular Expression (Regex): /^[a-f0-9]{16}$/ means: ^ (start of string), [a-f0-9] (only lowercase hex characters), {16} (exactly 16 characters long), $ (end of string). This prevents malicious input in route parameters.

⚠️ Junior Developer Trap / Common Gotcha

Omitting .trim() on input fields can allow users to submit titles consisting solely of blank spaces that pass length checks.

🎯 Technical Interview Pro-Tip

Q: Why validate query parameters like "k" and "threshold" in semantic search?
A: k controls how many results are returned. If a user passed k=1000000, it could exhaust server memory. Setting .isInt({ min: 1, max: 20 }) protects system resources.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

createQuestionValidation

Lines 6-22

WHY:
Enforces title between 5-255 characters and content >= 10 characters.

WHEN:
Runs on POST /api/questions.

HOW:
body('title').trim().isLength({ min: 5, max: 255 });
body('content').trim().isLength({ min: 10 }).

getSingleQuestionsValidation

Lines 38-45

WHY:
Validates that questionHash matches the exact 16-character hex format.

WHEN:
Runs on GET /api/questions/:questionHash.

HOW:
param('questionHash').matches(HASH_REGEX).

searchQuestionsSemanticValidation

Lines 47-66

WHY:
Validates semantic search queries: query >= 5 chars, k between 1 and 20, threshold between 0 and 1.

WHEN:
Runs on GET /api/questions/search.

HOW:
query('query').isLength({ min: 5 }); query('k').isInt({ min: 1, max: 20 }); query('threshold').isFloat({ min: 0, max: 1 }).

backend/src/api/question/controller/question.controller.js

149 lines

QUESTION REQUEST CONTROLLERS (JS) • QUESTION · CONTROLLER

❓ WHY IS IT HERE?

Bridges client requests with the question service and Gemini AI coaching service.

⏱️ WHEN DOES IT RUN?

Executes upon incoming HTTP requests to /api/questions.

⚙️ HOW DOES IT WORK?

Extracts req.body, req.params, and req.query, invokes services, and sends JSON response.

💡 Beginner Concept Spotlight

req.query vs req.params vs req.body: req.params is inside the URL path (/questions/:questionHash). req.query is after the question mark (/questions?search=react&mine=true). req.body is the JSON payload sent in POST/PUT requests.

⚠️ Junior Developer Trap / Common Gotcha

Query parameters from req.query are always STRINGS! If you pass req.query.k directly to a mathematical function without Number(req.query.k), you can introduce subtle calculation bugs.

🎯 Technical Interview Pro-Tip

Q: Why does createQuestionController pass req.user.id into the service instead of trusting an id sent in req.body?
A: Security! Users could tamper with req.body to impersonate someone else. req.user.id comes directly from the verified JWT token signature, guaranteeing authentic authorship.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

createQuestionController

Lines 14-31

WHY:

Extracts title and content, creates question with vector embedding, and returns 201 Created.

WHEN:

POST /api/questions.

HOW:

Awaits createQuestionWithVectorService({ userId: req.user.id, title, content }).

getQuestionController

Lines 33-50

WHY:
Filters questions by search keyword or "mine" flag for user-authored topics.

WHEN:
GET /api/questions?search=...&mine=true.

HOW:
Passes filters to getQuestionService() and responds with StatusCodes.OK.

searchQuestionsSemanticController

Lines 67-84

WHY:
Performs AI vector similarity search using Gemini embeddings.

WHEN:
GET /api/questions/search?query=...&k=5.

HOW:
Invokes searchQuestionsSemanticService({ query, k, threshold }).

generateQuestionDraftCoachController

Lines 106-122

WHY:
Provides real-time drafting advice to learners before they publish a question.

WHEN:
POST /api/questions/draft-coach.

HOW:
Calls generateQuestionDraftCoachService({ title, content }) and returns tips array.

backend/src/api/question/service/question.service.js

303 lines

QUESTION CORE SERVICE (JS) • QUESTION · SERVICE

❓ WHY IS IT HERE?

The core business logic for questions: generates random 16-hex hashes, inserts rows, triggers vector embedding generation, and runs dynamic SQL queries with joins.

⏱️ WHEN DOES IT RUN?

Called by question controllers during question operations.

⚙️ HOW DOES IT WORK?

Uses crypto.randomBytes(8).toString('hex') for question hashes; executes complex SQL JOINs across users and answers tables.

💡 Beginner Concept Spotlight

SQL LEFT JOIN vs INNER JOIN: An INNER JOIN only returns rows if there is a match in both tables. A LEFT JOIN returns ALL questions even if they have zero answers! That is why LEFT JOIN answers is used here so unanswered questions still appear in the feed.

⚠️ Junior Developer Trap / Common Gotcha

When counting answers with LEFT JOIN, always use COUNT(DISTINCT a.answer_id) instead of COUNT(*), otherwise joining multiple tables will multiply your counts!

🎯 Technical Interview Pro-Tip

Q: What happens if vector generation fails when creating a question?
A: Notice lines 70-82: The question creation is resilient! If the Gemini API fails or times out, the question record is still safely saved in MySQL, and a vector record is saved with status='failed'. The user never loses their post!

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Cryptographic Hash Generator

Lines 13

```
const generateQuestionHash = () => crypto.randomBytes(8).toString('hex');
```

WHY:
Creates an unpredictable, URL-safe 16-character identifier.

WHEN:
Called during question insertion.

HOW:
Generates 8 random bytes and converts to hexadecimal string.

createQuestionWithVectorService

Lines 15-87

WHY:
Inserts question, generates vector embedding via Gemini, and saves to question_vectors.

WHEN:
Called when posting a new question.

HOW:
safeExecute(INSERT INTO questions) → generateQuestionEmbedding(sourceText) → startQuestionVector().

buildQuestionFilters & getQuestionService

Lines 89-165

WHY:
Dynamically builds WHERE conditions for keyword searches and user-only filters, grouping with COUNT(DISTINCT a.answer_id).

WHEN:
Called to populate the dashboard feed.

HOW:
Constructs dynamic SQL string: WHERE q.title LIKE ? OR q.content LIKE ? with ORDER BY created_at DESC.

getSingleQuestionService

Lines 167-248

WHY:
Retrieves question details along with all nested community answers and author information.

WHEN:
Called when opening /questions/:questionHash.

HOW:
Queries questions table by question_hash, then executes second query for answers joined with author names.

backend/src/api/question/service/vector.service.js

460 lines

AI VECTOR & COSINE SIMILARITY SERVICE (JS) • QUESTION · VECTOR / AI

❓ WHY IS IT HERE?

The mathematical AI engine. Converts human language into geometric vectors and calculates cosine similarity between questions.

⏱️ WHEN DOES IT RUN?

Runs on question creation, semantic search, and related question lookups.

⚙️ HOW DOES IT WORK?

Invokes @google/genai embedContent(), validates arrays, calculates dot product divided by vector norms, and ranks results.

💡 Beginner Concept Spotlight

What is Cosine Similarity? Imagine each question is an arrow pointing in 1,536-dimensional space. Cosine similarity measures the angle between two arrows: 1.0 means identical direction (same meaning), 0 means perpendicular (unrelated). Even if two questions use completely different words (e.g. "car" vs "automobile"), their vectors point in the same direction!

⚠️ Junior Developer Trap / Common Gotcha

Vector searches will fail if the Gemini API key is invalid or quota is exceeded. Notice lines 217-322 contain fallbackKeywordSearch() so search NEVER breaks for users!

🎯 Technical Interview Pro-Tip

Q: How is Cosine Similarity mathematically calculated in JavaScript?
A: Dot Product: sum(a[i] * b[i]). Magnitudes: sqrt(sum(a[i]^2)) and sqrt(sum(b[i]^2)). Cosine Similarity = dotProduct / (normA * normB).

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

generateQuestionEmbedding Function

Lines 28-56

WHY:

Calls Google Gemini API to translate question text into a 1,536-dimensional vector array.

WHEN:

Invoked whenever text needs to be embedded.

HOW:

Calls genAI.models.embedContent({ model: "gemini-embedding-001", contents: sourceText, config: { outputDimensionality: 1536 } }).

calculateCosineSimilarity Pure Function

Lines 145-160

```
function calculateCosineSimilarity(vecA, vecB) {
  if (!vecA || !vecB || vecA.length === 0 || vecB.length === 0) return 0;
  const len = Math.min(vecA.length, vecB.length);
  let dotProduct = 0, normA = 0, normB = 0;
  for (let i = 0; i < len; i++) {
    dotProduct += vecA[i] * vecB[i];
    normA += vecA[i] * vecA[i];
    normB += vecB[i] * vecB[i];
  }
  const magnitude = Math.sqrt(normA) * Math.sqrt(normB);
  return magnitude === 0 ? 0 : dotProduct / magnitude;
}
```

WHY:
Computes the mathematical angle between two semantic concept vectors.

WHEN:
Executed in a loop comparing query embedding against all stored question embeddings.

HOW:
Calculates dot product and Euclidean norms, returning a similarity score between 0.0 and 1.0.

fallbackKeywordSearch Function

Lines 217-322

WHY:
Graceful degradation: if AI API is down, automatically switches to intelligent SQL keyword token matching.

WHEN:
Triggered if no vector results are found or Gemini API is offline.

HOW:
Splits query into words >= 3 characters, filters stop words, and executes dynamic SQL LIKE ? queries.

findSimilarQuestionByText Function

Lines 324-371

WHY:
Top-level semantic search coordinator.

WHEN:
Called during /api/questions/search.

HOW:
Generates query embedding → fetches ready embeddings → computes cosine similarity → sorts descending → hydrates question details from SQL.

backend/src/api/question/service/geminiTextCoach.service.js

129 lines

GEMINI TEXT COACH & ANSWER FIT SERVICE (JS) • QUESTION · GEMINI AI

❓ WHY IS IT HERE?

Acts as an AI mentor. Guides learners to write reproducible, context-rich questions and evaluates whether answer drafts actually answer the question.

⏱️ WHEN DOES IT RUN?

Called when clicking "AI suggestions" while composing a question, or "Check draft fit" while answering.

⚙️ HOW DOES IT WORK?

Sends engineered prompts to Gemini Flash requesting strict JSON responses, and safely parses JSON without markdown fences.

💡 Beginner Concept Spotlight

Prompt Engineering with Strict JSON: LLMs naturally want to talk in chatty prose. By providing an explicit JSON schema in the prompt (e.g. {"tips": ["...", "..."]}) and stripping markdown code fences (```json), we can reliably consume AI output in our code.

⚠️ Junior Developer Trap / Common Gotcha

LLMs sometimes wrap JSON in markdown fences (```json ... ```). Notice parseJsonObjectFromGeminiText() strips these fences with regex before calling JSON.parse().

🎯 Technical Interview Pro-Tip

Q: Why use gemini-2.5-flash-lite / gemini-1.5-flash for draft coaching instead of larger reasoning models?
A: Latency and cost! Draft coaching runs interactively while the user is typing. Flash models respond in under 1 second with near-zero token cost while providing excellent formatting feedback.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

parseJsonObjectFromGeminiText Helper

Lines 17-30

WHY:
Cleans and parses raw AI output into a native JavaScript object.

WHEN:
Runs on every LLM text response.

HOW:
Strips ```json fences with regex and runs JSON.parse() safely inside try/catch.

generateQuestionDraftCoachService

Lines 49-87

WHY:
Critiques a question draft and provides 3-5 constructive tips for peer readability.

WHEN:
Called from the Post Question page.

HOW:
Prompts Gemini Flash with question title and body, extracting checklist tips (error messages, repro steps, expected vs actual).

assessAnswerAgainstQuestionService

Lines 89-128

WHY:
Assesses whether an answer draft engages with the question before posting.

WHEN:
Called from the Question Detail reply form.

HOW:
Prompts Gemini Flash to evaluate relevance ("strong" | "partial" | "weak") and returns an objective one-sentence note.

SECTION 05 // SYSTEM ARCHITECTURE



Community answers, anti-self-answering business rules, and answer retrieval.

backend/src/api/answer/routes/answer.routes.js

10 lines

ANSWER ROUTE ENDPOINTS (JS) • ANSWER · ROUTE

❓ WHY IS IT HERE?

Defines the API endpoint for submitting solutions to questions.

⏱️ WHEN DOES IT RUN?

Mounted under /api/answers.

⚙️ HOW DOES IT WORK?

router.post('/', authenticateUser, createAnswerValidation, createAnswerController).

💡 Beginner Concept Spotlight

Protected Mutation Endpoints: Any endpoint that modifies data (creating an answer) must require authentication so every action is permanently attributed to a verified user account.

⚠️ Junior Developer Trap / Common Gotcha

Remember that answer routes are mounted under /api/answers in routes.js, so router.post('/') responds to POST /api/answers.

🎯 Technical Interview Pro-Tip

Q: Why not put answers under /api/questions/:questionId/answers?
A: Both are valid REST patterns. /api/questions/:questionId/answers is nested REST; POST /api/answers with { questionId } in the body is flatter and simplifies client-side state management when posting.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Answer Route Definition

Lines 1-10

WHY:

Secures and routes answer submissions.

WHEN:

POST /api/answers.

HOW:

Chains authenticateUser → createAnswerValidation → createAnswerController.

backend/src/api/answer/validations/answer.validation.js

23 lines

ANSWER INPUT VALIDATION (JS) • ANSWER · VALIDATION

❓ WHY IS IT HERE?

Guarantees answers are attached to valid questions and prevents spam or blank answers.

⏱️ WHEN DOES IT RUN?

Executes before createAnswerController.

⚙️ HOW DOES IT WORK?

Checks questionId.isInt({ min: 1 }) and content.isLength({ min: 20 }).

💡 Beginner Concept Spotlight

Meaningful Content Thresholds: Requiring a minimum of 20 characters prevents unhelpful one-word answers (e.g. "thanks", "bump", "same") and encourages descriptive explanations.

⚠️ Junior Developer Trap / Common Gotcha

Always convert string IDs to integers using .toInt() in express-validator so services receive clean integer types.

🎯 Technical Interview Pro-Tip

Q: What happens if a user submits an answer with 19 characters?
A: The express-validator rejects it with "Answer content must be at least 20 characters long", returning HTTP 400 Bad Request before hitting the database.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

createAnswerValidation Schema

Lines 4-22

WHY:

Validates question ID and enforces 20+ character minimum answer length.

WHEN:

POST /api/answers.

HOW:

body('questionId').notEmpty().isInt({ min: 1 }).toInt();
body('content').isLength({ min: 20 });

backend/src/api/answer/controller/answer.controller.js

23 lines

ANSWER CONTROLLER (JS) • ANSWER · CONTROLLER

❓ WHY IS IT HERE?

Unpacks questionId and content, attaches the logged-in userId, and returns HTTP 201 Created.

⏱️ WHEN DOES IT RUN?

Called upon submitting an answer.

⚙️ HOW DOES IT WORK?

Awaits createAnswerService and returns res.status(201).json({ success: true, data: answer }).

💡 Beginner Concept Spotlight

Attaching Authenticated User Context: Notice req.user.id is passed into createAnswerService. The client never provides their user ID in the body—the server reads it from the verified token.

⚠️ Junior Developer Trap / Common Gotcha

Ensure you return HTTP 201 Created for new answers rather than 200 OK to comply with REST standards.

🎯 Technical Interview Pro-Tip

Q: How does the controller handle errors thrown in the service?
A: Via try/catch block passing error to next(error), which invokes our centralized errorHandler middleware.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

createAnswerController Implementation

Lines 4-22

WHY:
Receives answer submission and delegates to service layer.

WHEN:
POST /api/answers.

HOW:
createAnswerService({ questionId, content, userId: req.user.id }) → res.status(201).

backend/src/api/answer/service/answer.service.js

62 lines

ANSWER BUSINESS SERVICE (JS) • ANSWER · SERVICE

❓ WHY IS IT HERE?

Core business logic for answers: verifies question existence, checks ownership, prevents self-answering, inserts the record, and hydrates author info.

⏱️ WHEN DOES IT RUN?

Invoked by answer controller.

⚙️ HOW DOES IT WORK?

Queries question owner; if question.user_id === userId, throws BadRequestError; inserts answer and calls getSingleAnswerService().

💡 Beginner Concept Spotlight

Enforcing Business Rules in Services: Business rules (like "a student cannot answer their own question") belong in the service layer because they represent business policy, not HTTP routing.

⚠️ Junior Developer Trap / Common Gotcha

When returning the newly inserted answer, don't just return { id: insertId }! Query the answer and join with the users table so the frontend can immediately display the author's name without a page refresh.

🎯 Technical Interview Pro-Tip

Q: Why is preventing users from answering their own questions an important business rule?
A: In study forums, self-answering is often used to bump threads or bypass discussion. Authors should edit their question content or reply in the comments, keeping answers reserved for peers and mentors.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

getQuestionOwner Helper

Lines 17-26

WHY:
Looks up the author ID of a question to verify ownership.

WHEN:
Runs before answer insertion.

HOW:
SELECT user_id FROM questions WHERE question_id = ?. Throws NotFoundError if question does not exist.

createAnswerService with Anti-Self-Answering Check

Lines 28-38

WHY:
Blocks users from answering their own questions and inserts valid answers.

WHEN:
POST /api/answers.

HOW:
if (question.user_id === userId) throw new BadRequestError('You cannot answer your own question'); INSERT INTO answers...

getSingleAnswerService & mapAnswer

Lines 40-61

WHY:
Hydrates a newly created answer with author first and last name for instant UI rendering.

WHEN:
Called immediately after answer insertion.

HOW:
SELECT answers JOIN users ON users.user_id = answers.user_id WHERE answer_id = ?.

SECTION 06 // SYSTEM ARCHITECTURE



PDF uploads with Multer, text extraction with pdf-parse, chunk embeddings, and conversational AI document query.

backend/src/api/rag/config/rag.upload.config.js

55 lines

MULTER STORAGE & UPLOAD CONFIG (JS) • RAG · UPLOAD CONFIG

❓ WHY IS IT HERE?

Safely receives binary PDF files over HTTP, isolates files by user ID, and prevents malicious non-PDF uploads.

⏱️ WHEN DOES IT RUN?

Executes when a multipart/form-data request is received at POST /api/rag/documents.

⚙️ HOW DOES IT WORK?

Uses multer.diskStorage(), creates per-user directory paths, generates timestamped unique filenames, and validates application/pdf mime-type.

💡 Beginner Concept Spotlight

What is Multipart/Form-Data? Standard HTTP requests send text JSON. When uploading binary files (PDFs, images), the browser splits the payload into parts separated by boundaries. Multer parses these binary chunks and writes them to disk.

⚠️ Junior Developer Trap / Common Gotcha

Never store files with their original raw filenames! Malicious filenames (like ../../etc/passwd or test.php) can overwrite system files. Notice how sanitizedOriginal replaces non-alphanumeric characters with underscores.

🎯 Technical Interview Pro-Tip

Q: Why isolate file uploads into per-user directory folders (uploads/rag/<userId>)?
A: Multi-tenant security and filesystem efficiency. If two users upload "syllabus.pdf", they will not collide. It also simplifies deleting all files when a user account is removed.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

multer.diskStorage Configuration

Lines 14-27

WHY: Defines storage destination and unique sanitized filenames.

WHEN: Runs on file upload.

HOW: path.join(uploadDir, String(req.user?.id)); filename uses Date.now() + random number + sanitized original name.

fileFilter Mime-Type Guard

Lines 29-35

WHY:
Restricts uploads strictly to valid PDF documents.

WHEN:
Inspects file headers before writing to disk.

HOW:
Checks file.mimetype === 'application/pdf' and .endsWith('.pdf'). Rejects all other extensions.

createDocumentMulterErrorHandler

Lines 43-54

WHY:
Translates Multer file size limit errors into clean HTTP 400 responses.

WHEN:
Triggered if a file exceeds RAG_MAX_UPLOAD_MB (e.g. 10MB).

HOW:
Checks err instanceof multer.MulterError and LIMIT_FILE_SIZE.

backend/src/api/rag/routes/rag.routes.js

73 lines

RAG DOCUMENT ROUTES (JS) • RAG · ROUTE

❓ WHY IS IT HERE?

Exposes the full suite of document intelligence endpoints.

⏱️ WHEN DOES IT RUN?

Mounted under /api/rag/documents.

⚙️ HOW DOES IT WORK?

Binds routes with authenticateUser, multer middleware, validators, and controllers.

💡 Beginner Concept Spotlight

Streaming Binary Files: Notice GET /:documentId/file. Instead of returning JSON, this endpoint streams the raw PDF binary directly to the browser so the built-in PDF reader can render it.

⚠️ Junior Developer Trap / Common Gotcha

Make sure delete routes also delete physical files from the hard drive, not just database rows, otherwise orphaned files will consume all disk space over time.

🎯 Technical Interview Pro-Tip

Q: Why does POST / return HTTP 202 Accepted instead of HTTP 200 OK?
A: HTTP 202 Accepted signals that the server has accepted the upload for processing, but the heavy work (extracting text and generating vector embeddings) is happening asynchronously in the background.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

List and Upload Routes

Lines 25-34

WHY:

Lists a user's documents and accepts new PDF file uploads.

WHEN:

Navigating to Knowledge Base or uploading files.

HOW:

GET / → listDocumentsController; POST / → ragUpload.single('file') → uploadDocumentController.

Search, Query, and File Stream Sub-Routes

Lines 37-56

WHY:
Enables semantic search in documents, AI conversational Q&A, and inline PDF preview.

WHEN:
User searches passages, asks AI questions, or previews PDF.

HOW:
GET /:documentId/search, POST /:documentId/query, GET /:documentId/file.

backend/src/api/rag/validations/rag.validation.js

54 lines

RAG VALIDATION SCHEMAS (JS) • RAG · VALIDATION

❓ WHY IS IT HERE?

Validates that document IDs are positive integers and natural language queries are non-empty.

⏱️ WHEN DOES IT RUN?

Runs before RAG controller handlers.

⚙️ HOW DOES IT WORK?

Uses param('documentId').isInt({ min: 1 }) and custom query body checks.

💡 Beginner Concept Spotlight

Custom Validators: express-validator allows .custom((value, { req }) ⇒ ...) to perform dynamic multi-field checks (e.g. checking whether query was passed in body as query or question).

⚠️ Junior Developer Trap / Common Gotcha

Always ensure search queries are at least 2 characters long to avoid running expensive vector embeddings on empty strings.

🎯 Technical Interview Pro-Tip

Q: How do validation schemas protect downstream LLM API quotas?
A: By catching invalid, empty, or malicious queries before they trigger Gemini API calls, saving both bandwidth and token expenses.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

documentIdParamValidation

Lines 4-12

WHY: Ensures :documentId in URL is a valid positive integer.

WHEN: Runs on all /:documentId routes.

HOW: param('documentId').notEmpty().isInt({ min: 1 }).toInt().

searchDocumentValidation

Lines 21-34

WHY:
Ensures search query is at least 2 characters and k is between 1 and 50.

WHEN:
GET /:documentId/search.

HOW:
Validates query text and caps top-k results to 50.

backend/src/api/rag/controller/rag.controller.js

144 lines

RAG DOCUMENT CONTROLLERS (JS) • RAG · CONTROLLER

❓ WHY IS IT HERE?

Orchestrates file upload responses, document deletion, PDF binary streaming, and query execution.

⏱️ WHEN DOES IT RUN?

Executes upon incoming requests to /api/rag/documents.

⚙️ HOW DOES IT WORK?

Calls rag.service.js functions, sets Content-Type: application/pdf for streaming, and returns JSON.

💡 Beginner Concept Spotlight

Content-Disposition: inline vs. attachment: Setting Content-Disposition: inline tells the browser to display the PDF directly inside an <iframe>. Setting attachment forces the browser to download the file to the user's computer.

⚠️ Junior Developer Trap / Common Gotcha

Always verify document ownership (assertOwnedDocument) before streaming or deleting files! Otherwise, User A could guess User B's documentId and read their private files!

🎯 Technical Interview Pro-Tip

Q: How does the server prevent IDOR (Insecure Direct Object Reference) vulnerabilities in document downloads?
A: By checking WHERE document_id = ? AND user_id = ? in assertOwnedDocument. Even if an attacker knows a documentId, the database will return NotFoundError if it belongs to another user.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

uploadDocumentController (HTTP 202 Accepted)

Lines 15-30

WHY:

Accepts upload, saves file, and immediately returns 202 so frontend can start polling.

WHEN:

POST /api/rag/documents.

HOW:

Awaits createDocumentFromUploadService and responds with StatusCodes.ACCEPTED (202).

getDocumentFileController (PDF Streamer)

Lines 63-84

WHY:
Streams the physical PDF from disk into the browser reader.

WHEN:
GET /api/rag/documents/:documentId/file.

HOW:
assertOwnedDocument → verifies file exists → res.setHeader('Content-Type', 'application/pdf') → res.sendFile(fullPath).

searchDocumentController & queryDocumentController

Lines 103-143

WHY:
Handles passage retrieval and grounded AI answer generation.

WHEN:
User searches text or asks an AI question in the Knowledge Base.

HOW:
Calls searchInDocumentService and queryDocumentService, returning excerpts and citations.

backend/src/api/rag/service/rag.service.js

433 lines

RAG INGESTION, CHUNKING & QUERY ENGINE (JS) • RAG · SERVICE

❓ WHY IS IT HERE?

The complete RAG brain of the application. Takes a raw PDF file, extracts text, breaks it into semantic chunks, generates embeddings, finds matching chunks using cosine similarity, and builds grounded AI prompts.

⏱️ WHEN DOES IT RUN?

Runs in the background after PDF upload, and synchronously when querying documents.

⚙️ HOW DOES IT WORK?

Uses pdf-parse, sliding character window algorithm, Gemini embeddings (1536 dims), cosine similarity, and Gemini Flash for grounded text generation.

💡 Beginner Concept Spotlight

Why RAG (Retrieval-Augmented Generation)? LLMs have a knowledge cutoff and do not know your private notes. Instead of fine-tuning a model (expensive), RAG finds the exact relevant paragraphs from your PDF and injects them directly into the prompt: "Answer this question using ONLY these excerpts".

⚠️ Junior Developer Trap / Common Gotcha

Background processing with setImmediate(): Heavy PDF parsing must NEVER be awaited in the HTTP upload controller, otherwise client requests will time out on large PDFs. setImmediate() delegates work to the next tick of the Node.js event loop.

🎯 Technical Interview Pro-Tip

Q: How does the chunking algorithm avoid cutting words in half?
A: In chunkText(), when reaching the target chunkSize (900 chars), the algorithm looks backwards for the nearest space with text.indexOf(' ', endIndex - 40). If found, it breaks at the space boundary instead of splitting a word.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

extractTextFromPdfBuffer Function

Lines 65-81

WHY:
Extracts plain text from raw PDF binary buffers.

WHEN:
Called during background document processing.

HOW:
Loads pdf-parse dynamically, extracts textResult.text, and destroys parser instance.

chunkText Sliding Window Algorithm

Lines 83-123

WHY:
Splits long documents into manageable, overlapping 900-character chunks with word-boundary awareness.

WHEN:
Runs after text extraction.

HOW:
Loop advances by (chunkSize - overlap). Snaps to nearest space index to prevent broken words.

processDocumentInBackground Worker

Lines 197-245

WHY:
Asynchronously reads PDF, extracts text, chunks it, generates vector embeddings for every chunk, and updates status to "ready".

WHEN:
Dispatched via setImmediate() immediately after file upload.

HOW:
Loops through chunks, calls generateEmbedding(), stores into document_chunks and document_chunk_vectors, then sets status='ready'.

searchInDocumentService Function

Lines 285-367

WHY:
Finds the top-K most relevant chunks within a specific document matching a search query.

WHEN:
Called during document search or AI Q&A.

HOW:
Generates query embedding → computes cosine similarity against document chunk vectors → sorts descending → filters by threshold (0.45).

queryDocumentService Function (Grounded Q&A)

Lines 369-432

WHY:
Retrieves relevant excerpts and instructs Gemini to answer strictly based on those excerpts with citations.

WHEN:
Called when a user asks a question about a PDF.

HOW:
Combines top chunks into DOCUMENT EXCERPTS, prompts Gemini Flash, and returns { answer, citations, chunksUsed }.

SECTION 07 // SYSTEM ARCHITECTURE



Vite build tool, HTML entry, styling tokens, application routing, and utility helpers.

frontend/package.json

42 lines

FRONTEND MANIFEST (JSON) • CONFIG · FRONTEND

❓ WHY IS IT HERE?

Declares client dependencies and build tooling for the React Single Page Application.

⏱️ WHEN DOES IT RUN?

Used by npm and Vite during build and development.

⚙️ HOW DOES IT WORK?

Defines Vite scripts: "dev", "build", "preview", and "test" via vitest.

💡 Beginner Concept Spotlight

SPA (Single Page Application): Instead of the browser refreshing and downloading a new HTML page on every click, React downloads one HTML file and dynamically swaps components using JavaScript, making page transitions instant.

⚠️ Junior Developer Trap / Common Gotcha

Lucide-react icons and Framer Motion must be compatible with your React version. ForumStudy uses React 19 with compatible modern releases.

🎯 Technical Interview Pro-Tip

Q: Why is Vite significantly faster than traditional Webpack for development?
A: Webpack bundles the entire application before starting the server. Vite uses native browser ES Modules (ESM) during development, serving files on demand and providing near-instant Hot Module Replacement (HMR).

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Frontend Production Dependencies

Lines 15-23

WHY:

React 19 powers the component tree; React Router 7 manages navigation; Axios handles HTTP; Lucide provides icons; Framer Motion enables smooth animations.

WHEN:

Bundled into the client JavaScript build.

HOW:

Resolved by Vite during development and production rollup bundling.

frontend/vite.config.js

8 lines

VITE CONFIGURATION (JS) • CONFIG · VITE

❓ WHY IS IT HERE?

Tells Vite to use the official @vitejs/plugin-react to transform JSX into optimized JavaScript.

⏱️ WHEN DOES IT RUN?

Read by Vite CLI on vite dev or vite build.

⚙️ HOW DOES IT WORK?

defineConfig({ plugins: [react()] }).

💡 Beginner Concept Spotlight

JSX Compilation: Browsers cannot read HTML tags inside JavaScript files (JSX). Vite's React plugin compiles JSX syntax like <h1>Hello</h1> into React.createElement('h1', null, 'Hello').

⚠️ Junior Developer Trap / Common Gotcha

Without the React plugin, attempting to import .jsx files will result in syntax errors during build.

🎯 Technical Interview Pro-Tip

Q: What is Fast Refresh in @vitejs/plugin-react?
A: Fast Refresh allows developers to edit React components and see updates in the browser in milliseconds without losing component state (e.g. form inputs remain filled).

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Vite Plugin Config

Lines 1-8

WHY:
Enables React JSX transformation and Fast Refresh.

WHEN:
Loaded on Vite startup.

HOW:
export default defineConfig({ plugins: [react()] });

frontend/index.html

14 lines

HTML APPLICATION SHELL (HTML) • FRONTEND · HTML

❓ WHY IS IT HERE?

The single HTML shell loaded by the browser where the entire React component hierarchy is mounted.

⏱️ WHEN DOES IT RUN?

Loaded first when visiting the website.

⚙️ HOW DOES IT WORK?

Provides   and loads /src/main.jsx via <script type="module">.

💡 Beginner Concept Spotlight

The Single Div (#root): In a React SPA, the entire application (hundreds of components, modals, feeds) lives inside a single . React manipulates this div dynamically.

⚠️ Junior Developer Trap / Common Gotcha

Always include <meta name="viewport" content="width=device-width, initial-scale=1.0" />, otherwise mobile devices will render the website as a zoomed-out desktop page.

🎯 Technical Interview Pro-Tip

Q: Why does the script tag in Vite use type="module"?
A: It instructs the browser to treat src/main.jsx as a native ES Module, enabling the browser itself to resolve imports asynchronously without waiting for a monolithic bundle.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

HTML Skeleton & Root Mount Point

Lines 1-14

WHY:

Defines viewport meta tags, page title, and loads main.jsx into #root.

WHEN:

Initial browser page load.

HOW:

; <script type="module" src="/src/main.jsx"></script>

frontend/src/main.jsx

11 lines

REACT ENTRY POINT (JSX) • FRONTEND · MAIN

❓ WHY IS IT HERE?

Initializes the React 19 runtime, imports global styles, and attaches the <App /> component to the DOM.

⏱️ WHEN DOES IT RUN?

Executes immediately when index.html loads script.

⚙️ HOW DOES IT WORK?

Calls createRoot(document.getElementById('root')).render(<StrictMode><App /></StrictMode>).

💡 Beginner Concept Spotlight

StrictMode: A development-only tool in React that deliberately runs components and effects twice to help developers detect side-effects, memory leaks, and deprecated APIs.

⚠️ Junior Developer Trap / Common Gotcha

Seeing console.log fire twice in your React components is NORMAL in development! It is StrictMode ensuring your components are pure functions.

🎯 Technical Interview Pro-Tip

Q: What is the advantage of createRoot in React 18/19 over ReactDOM.render?
A: createRoot enables Concurrent Mode, allowing React to interrupt, pause, and prioritize high-priority rendering tasks (like user typing) over background renders.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

createRoot Mounting

Lines 6-10

WHY:
Attaches the React tree to the physical DOM.

WHEN:
Browser initialization.

HOW:
createRoot(document.getElementById('root')).render(<StrictMode><App /></StrictMode>).

frontend/src/App.jsx

89 lines

APPLICATION ROUTE MAP (JSX) • FRONTEND · APP / ROUTES

❓ WHY IS IT HERE?

The master routing blueprint of the frontend. Separates public marketing pages from protected authenticated forum tools.

⏱️ WHEN DOES IT RUN?

Renders whenever the URL path changes.

⚙️ HOW DOES IT WORK?

Uses BrowserRouter, Routes, Route, and Navigate from react-router-dom.

💡 Beginner Concept Spotlight

Route Nesting: Notice how protected routes are wrapped inside <Route element={<Layout />}>. This means the Sidebar, Navbar, and Footer stay rendered while only the center content swaps out!

⚠️ Junior Developer Trap / Common Gotcha

Always include a catch-all route (<Route path="*" element={<Navigate to="/" replace />} />) so mistyped URLs redirect gracefully instead of showing a blank screen.

🎯 Technical Interview Pro-Tip

Q: How does ProtectedRoute work in App.jsx?
A: ProtectedRoute is a wrapper component that checks if the user is logged in. If yes, it renders children; if no, it redirects to /auth while preserving the attempted URL.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Public Routes (Landing & Auth)

Lines 24-27

WHY:
Accessible to visitors without needing an account or login token.

WHEN:
Visiting / or /auth.

HOW:
<Route path="/" element={<Landing />} />; <Route path="/auth" element={<Auth />} />

Protected Layout Routes

Lines 29-78

WHY:
Renders Dashboard, PostQuestion, MyQuestions, QuestionDetail, and RagDocuments inside the shared Layout shell guarded by ProtectedRoute.

WHEN:
Authenticated user navigation.

HOW:
Wraps routes inside <Layout /> and <ProtectedRoute>.

Catch-All Redirect

Lines 80-82

WHY:
Prevents broken links by redirecting unknown paths to home.

WHEN:
Navigating to an undefined URL.

HOW:
<Route path="*" element={<Navigate to="/" replace />} />

frontend/src/index.css

229 lines

GLOBAL DESIGN SYSTEM & TOKENS (CSS) • FRONTEND · GLOBAL CSS

❓ WHY IS IT HERE?

The visual design system of ForumStudy. Defines reusable CSS variables for colors, shadows, border radii, and spacing, ensuring 100% visual consistency.

⏱️ WHEN DOES IT RUN?

Loaded globally across all pages and components.

⚙️ HOW DOES IT WORK?

Defines :root CSS custom properties and standard resets (* { box-sizing: border-box }).

💡 Beginner Concept Spotlight

CSS Custom Properties (--variables): Defining colors like --primary: #f97316 allows changing the entire application theme in one central place rather than hunting through 50 separate CSS files.

⚠️ Junior Developer Trap / Common Gotcha

Always set box-sizing: border-box in global resets! Without it, adding padding to an element expands its width, breaking responsive grid layouts.

🎯 Technical Interview Pro-Tip

Q: What is the benefit of using CSS variables over Sass variables?
A: Sass variables are compiled away into static values at build time. CSS variables exist in the runtime browser DOM, meaning they can be modified dynamically with JavaScript or media queries for instant theme switching.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Color Palette Tokens

Lines 16-60

WHY:

Tailored palette: Primary orange, slate neutrals, surface containers, status colors (emerald, amber, red, blue).

WHEN:

Referenced across all component stylesheets.

HOW:

--primary: #f97316; --secondary: #475569; --surface: #ffffff; etc.

Elevation, Radius, Spacing, & Typography Tokens

Lines 65-125

WHY:
Standardizes shadow depths, border radii, font stacks (Inter), and layout widths (--app-content-max: 80rem).

WHEN:
Used to enforce responsive rhythm.

HOW:
--shadow-md, --radius-lg, --font-sans, --app-sidebar-width: 17rem.

Global Resets & Focus States

Lines 127-197

WHY:
Resets margins, sets smooth scrolling, and provides high-contrast accessible outline states on input focus.

WHEN:
Applies to all HTML elements.

HOW:
* { box-sizing: border-box; } input:focus { outline: none; border-color: var(--primary); }

frontend/src/lib/utils.js

106 lines

FRONTEND UTILITY HELPERS (JS) • FRONTEND · UTILS

❓ WHY IS IT HERE?

Encapsulates reusable UI helper functions: calculating "5 mins ago", formatting dates, extracting user initials, and generating deterministic avatar colors.

⏱️ WHEN DOES IT RUN?

Imported across question cards, headers, sidebars, and discussion threads.

⚙️ HOW DOES IT WORK?

Pure mathematical functions with string manipulation and bitwise hashing.

💡 Beginner Concept Spotlight

Pure Functions: A pure function always returns the exact same output for the same input and produces no side-effects (does not mutate global state). This makes utilities easy to test and debug.

⚠️ Junior Developer Trap / Common Gotcha

Always validate dates with isNaN(date.getTime()) before calling date methods to prevent "Invalid Date" from appearing in user interfaces.

🎯 Technical Interview Pro-Tip

Q: How does getAvatarColor() ensure a user always gets the exact same avatar color without saving it to the database?
A: It uses a deterministic string hash algorithm (bitwise hash = (hash << 5) - hash + charCode). The resulting integer modulo the palette array length always yields the same index for the same name!

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

timeAgo Relative Date Calculator

Lines 6-38

WHY:
Converts ISO timestamps into human-readable relative time ("just now", "5 mins ago", "2 days ago").

WHEN:
Used on question cards and answer timestamps.

HOW:
Computes elapsed seconds: Math.floor((now - date) / 1000) and branches into seconds, minutes, hours, days, weeks, months, or years.

getInitials Helper

Lines 58-64

WHY:
Extracts uppercase first letters from first and last names for avatar bubbles (e.g. "John Doe" → "JD").

WHEN:
Rendered in user avatar circles.

HOW:
firstName.trim().charAt(0).toUpperCase() + lastName.trim().charAt(0).toUpperCase().

getAvatarColor Deterministic Palette Hasher

Lines 71-91

WHY:
Assigns harmonious pastel background colors to user avatars consistently across all screens.

WHEN:
Called when rendering question authors or user profiles.

HOW:
Hashes name string with bitwise operators and selects from 8 tailored color pairs.

isAuthoredByUser Ownership Comparator

Lines 99-105

WHY:
Safely checks whether a question was created by the currently logged-in user.

WHEN:
Controls "YOURS" badge and prevents self-answering in the UI.

HOW:
Compares String(question.userId) === String(user.id) handling null/undefined.

frontend/src/styles/pageStates.module.css

70 lines

SHARED STATE PATTERNS (CSS MODULE) • FRONTEND · CSS MODULE

❓ WHY IS IT HERE?

Centralizes common UI states (Loading, Empty list, Error banners) so pages don't duplicate container styling.

⏱️ WHEN DOES IT RUN?

Applied when lists are loading, empty, or encountered a network failure.

⚙️ HOW DOES IT WORK?

BEM-styled classes using global design tokens.

💡 Beginner Concept Spotlight

CSS Modules: Unlike global CSS where class names can collide, CSS Modules automatically append unique hashes to class names (e.g. pageStates__message_a1b2c) during build, guaranteeing zero style leakage across components.

⚠️ Junior Developer Trap / Common Gotcha

In CSS Modules, use camelCase or bracket notation for class names with hyphens (e.g. styles['pageStates__message--error']).

🎯 Technical Interview Pro-Tip

Q: What are the UX benefits of having dedicated empty state designs?
A: Empty states prevent user confusion. Instead of displaying a blank white screen when no questions exist, an empty state explains why it is empty and provides a clear call-to-action ("Ask your first question!").

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Loading & Error State Classes

Lines 13-38

WHY:
Provides styled containers with subtle gradients for loading states and high-contrast red panels for errors.

WHEN:
Rendered conditionally based on loading/error state variables.

HOW:
.pageStates__message--loading and .pageStates__message--error.

Empty State Container

Lines 40-50

WHY:
Renders dashed border panels guiding users when no data exists.

WHEN:
questions.length === 0.

HOW:
border: 1px dashed var(--border); background-color: var(--surface-container-low).

SECTION 08 // SYSTEM ARCHITECTURE

Global authentication session management and typed API clients for questions, answers, auth, and RAG.

frontend/src/contexts/AuthContext.jsx

99 lines

AUTHENTICATION CONTEXT PROVIDER (JSX) • FRONTEND · CONTEXT

❓ WHY IS IT HERE?

Provides global user session state to the entire application so any component can check who is logged in without prop drilling.

⏱️ WHEN DOES IT RUN?

Wraps the entire application in App.jsx; maintains state across navigation.

⚙️ HOW DOES IT WORK?

Uses React createContext, useState, and useEffect to read stored credentials from localStorage on boot.

💡 Beginner Concept Spotlight

Prop Drilling vs. Context API: Prop drilling is passing a prop down through 5 layers of components just so a deep child can read it. Context API acts like a global broadcast tower: any component can tune in directly using the useAuth() hook!

⚠️ Junior Developer Trap / Common Gotcha

Always wrap localStorage access in try/catch or safety checks. In private browsing modes or restricted enterprise browsers, accessing localStorage can throw security exceptions.

🎯 Technical Interview Pro-Tip

Q: Why provide a custom useAuth() hook instead of exporting AuthContext directly?
A: Encapsulation and safety. useAuth() checks if useContext(AuthContext) is undefined. If a developer accidentally calls useAuth() outside of an AuthProvider, it throws an immediate descriptive error.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

AuthProvider & Session Restoration

Lines 13-29

WHY:
Restores user session from localStorage on initial page load to prevent unnecessary logouts on browser refresh.

WHEN:
Executes once on mount (useEffect with [] dependency).

HOW:
Reads authService.getStoredToken() and authService.getStoredUser(), updating user state.

register, login, and logout Methods

Lines 35-71

WHY:
Exposes asynchronous authentication methods that manage loading states and update session.

WHEN:
Called from login/register forms or navbar logout buttons.

HOW:
Awaits authService, updates user state, and navigates to /auth on logout.

useAuth Custom Hook

Lines 90-98

WHY:
The clean consumer interface for accessing authentication state anywhere in the app.

WHEN:
Called in components: const { user, logout } = useAuth();

HOW:
Calls useContext(AuthContext) and validates that context is defined.

frontend/src/services/core/api.client.js

55 lines

AXIOS HTTP CLIENT & INTERCEPTORS (JS) • FRONTEND · API CLIENT

❓ WHY IS IT HERE?

Centralized network highway. Eliminates repetitive fetch code, automatically attaches JWT auth headers to every request, and handles session expiry globally.

⏱️ WHEN DOES IT RUN?

Instantiated on app load; interceptors run on EVERY outgoing request and incoming response.

⚙️ HOW DOES IT WORK?

Uses axios.create(). Request interceptor reads localStorage.getItem('token') and sets config.headers.Authorization. Response interceptor catches status 401 and redirects to /auth.

💡 Beginner Concept Spotlight

HTTP Interceptors: Think of Axios interceptors like an automated mailroom. The outgoing interceptor stamps a security seal (JWT token) onto every letter before it leaves the building. The incoming interceptor checks if the response says "Access Denied" (401) and reroutes you to the security desk (login page).

⚠️ Junior Developer Trap / Common Gotcha

Notice lines 38-42: Always skip global 401 redirect for /api/auth/login! If a user enters an incorrect password, we want the login component to display "Invalid password", not refresh the entire page!

🎯 Technical Interview Pro-Tip

Q: What is the benefit of setting timeout: 10000 on an Axios client?
A: Without a timeout, a stalled server or lost connection can cause requests to hang forever, leaving buttons spinning and freezing the UI. A 10-second timeout ensures the app fails gracefully.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Axios Instance Setup

Lines 6-12

WHY:
Configures base URL from VITE_API_BASE_URL (defaults to http://localhost:3777) and 10s timeout.

WHEN:
Created once at startup.

HOW:
axios.create({ baseURL, timeout: 10000, headers: { 'Content-Type': 'application/json' } }).

Request Interceptor (Token Injection)

Lines 17-28

WHY:
Automatically attaches the user's Bearer token to all outgoing requests.

WHEN:
Fires right before any HTTP request leaves the browser.

HOW:
Reads localStorage token; sets config.headers.Authorization = `Bearer ${token}`.

Response Interceptor (Global 401 Catch-All)

Lines 33-53

WHY:
Detects expired tokens, clears stale localStorage data, and redirects to login.

WHEN:
Fires when a server response arrives with an error.

HOW:
If status === 401 and not an auth endpoint: removes token and user, window.location.href = '/auth'.

frontend/src/services/auth/auth.service.js

114 lines

FRONTEND AUTH SERVICE (JS) • FRONTEND · SERVICE

❓ WHY IS IT HERE?

Isolates authentication API endpoints and translates server error codes into friendly user messages.

⏱️ WHEN DOES IT RUN?

Called by AuthContext during authentication actions.

⚙️ HOW DOES IT WORK?

Uses apiClient.post(), saves tokens to localStorage, and normalizes errors with handleAuthError().

💡 Beginner Concept Spotlight

Centralized Error Translation: Backend errors can arrive in multiple shapes (error.response.data.msg or .message). A helper function normalizes them so components always receive a standard Error object with clean text.

⚠️ Junior Developer Trap / Common Gotcha

Always wrap JSON.parse(localStorage.getItem('user')) in try/catch! If corrupted data enters localStorage, JSON.parse can crash the entire application.

🎯 Technical Interview Pro-Tip

Q: What happens if an API request fails due to a network outage instead of a server response?
A: Notice lines 76-83: If error.response is undefined, the service checks if error.code === "ECONNABORTED" (timed out) or returns "Unable to connect to server. Please check your internet connection."

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

register & login Methods

Lines 7-32

WHY:
Sends credentials to backend, stores returned token and user in localStorage.

WHEN:
User clicks Register or Sign In.

HOW:
apiClient.post('/api/auth/login') → saves token and JSON user → returns { user, token }.

handleAuthError Centralized Normalizer

Lines 75-101

WHY:

Translates HTTP status codes (400, 401, 500, timeouts) into clear, friendly error messages.

WHEN:

Invoked whenever an auth request rejects.

HOW:

Inspects error.response.status and maps to appropriate Error instance.

frontend/src/services/question/question.service.js

210 lines

FRONTEND QUESTION SERVICE (JS) • FRONTEND · SERVICE

❓ WHY IS IT HERE?

Encapsulates all question operations: keyword listing, semantic vector search, draft coaching, and answer fit checking.

⏱️ WHEN DOES IT RUN?

Called by Dashboard, PostQuestion, and QuestionDetail pages.

⚙️ HOW DOES IT WORK?

Sends GET and POST requests via apiClient with query params and body payloads.

💡 Beginner Concept Spotlight

API Abstraction Layer: UI components should not know whether an endpoint is GET /api/questions?search=react or something else. They simply call questionService.getQuestions({ search: "react" }), making the UI clean and decoupled.

⚠️ Junior Developer Trap / Common Gotcha

Notice how getQuestions() safely handles different response shapes (payload.data, payload.questions, or plain payload) using ternary checks to prevent "undefined is not an array" crashes.

🎯 Technical Interview Pro-Tip

Q: Why does findSimilarQuestions() have a titleFallback parameter?
A: Graceful degradation! If the vector endpoint (/similar) fails, lines 134-142 fall back to searchQuestionsSemantic(titleFallback), ensuring the user still gets recommendations.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

getQuestions API Call

Lines 29-57

WHY:
Fetches question feed with optional search query or "mine" user filter.

WHEN:
Dashboard and MyQuestions page loads.

HOW:
apiClient.get('/api/questions', { params }) → normalizes array structure.

searchQuestionsSemantic API Call

Lines 91-112

WHY: Queries the AI vector semantic search endpoint.

WHEN: User clicks "AI Search" button in the search bar.

HOW: apiClient.get('/api/questions/search', { params: { query } }).

generateQuestionDraftCoach API Call

Lines 169-178

WHY: Asks Gemini AI for real-time suggestions on a draft question.

WHEN: User clicks "AI suggestions" while composing a question.

HOW: apiClient.post('/api/questions/draft-coach', { title, content }) → returns tips array.

assessAnswerFit API Call

Lines 186-199

WHY: Evaluates how well an answer addresses the original question.

WHEN: User clicks "Check draft fit" before submitting a reply.

HOW: apiClient.post(`/api/questions/${questionHash}/answer-fit`, { answerText }).

frontend/src/services/answer/answer.service.js

44 lines

FRONTEND ANSWER SERVICE (JS) • FRONTEND · SERVICE

❓ WHY IS IT HERE?

Encapsulates answer submission requests and error normalization.

⏱️ WHEN DOES IT RUN?

Called from QuestionDetail page when submitting a solution.

⚙️ HOW DOES IT WORK?

apiClient.post('/api/answers', { questionId, content }).

💡 Beginner Concept Spotlight

Single Responsibility: Keeping answer API calls in their own dedicated service file mirrors our backend modularity and makes testing straightforward.

⚠️ Junior Developer Trap / Common Gotcha

Remember that answers require at least 20 characters, so answerService will catch and bubble any 400 Bad Request errors from the server.

🎯 Technical Interview Pro-Tip

Q: How does answerService handle API errors?
A: handleAnswerError() catches network disconnects, timeouts, or backend validation messages and wraps them in clean Error instances.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

postAnswer Function

Lines 29-39

WHY:

Posts an answer payload to the server.

WHEN:

User clicks "Post Your Answer".

HOW:

apiClient.post('/api/answers', { questionId, content }) → returns created answer.

frontend/src/services/rag/rag.service.js

240 lines

FRONTEND RAG SERVICE (JS) • FRONTEND · SERVICE

❓ WHY IS IT HERE?

Manages the complete frontend RAG workflow: multipart file uploading, polling status every 3 seconds until indexing completes, document deletion, and grounded conversational Q&A.

⏱️ WHEN DOES IT RUN?

Called from RagDocuments workspace page.

⚙️ HOW DOES IT WORK?

Uses FormData for file uploads, setInterval/setTimeout for status polling, and blob URLs for inline PDF preview.

💡 Beginner Concept Spotlight

Polling Pattern: Because processing a 50-page PDF can take 10-20 seconds, the server returns immediately with status="processing". The client polls the document status every 3 seconds (pollDocumentStatus) until the status transitions to "ready" or "failed".

⚠️ Junior Developer Trap / Common Gotcha

Always revoke Object URLs with URL.revokeObjectURL(url) when previewing PDFs, otherwise browser memory will leak over time.

🎯 Technical Interview Pro-Tip

Q: Why use FormData instead of JSON when uploading PDFs?
A: JSON is a text format. Binary files like PDFs cannot be directly represented in JSON without Base64 encoding (which increases file size by 33%). FormData sends pure multipart binary streams efficiently.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

uploadPdf Function

Lines 76-89

WHY:
Appends file to FormData and sends multipart/form-data POST request.

WHEN:
User selects PDF and clicks Upload.

HOW:
formData.append('file', file); apiClient.post('/api/rag/documents', formData, { headers: { 'Content-Type': 'multipart/form-data' } }).

pollDocumentStatus Asynchronous Poller

Lines 98-116

WHY: Repeatedly checks document status every 3 seconds until status is "ready" or "failed".

WHEN: Initiated immediately after file upload succeeds.

HOW: While loop with setTimeout promise delay, fetching GET /api/rag/documents/:id.

queryDocument Function

Lines 179-212

WHY: Submits user questions to the grounded RAG AI engine and receives answers with citations.

WHEN: User clicks "Ask AI" in the Knowledge Base.

HOW: apiClient.post(`/api/rag/documents/${documentId}/query`, { query }) → returns { answer, citations }.

fetchPdfObjectUrl Function

Lines 219-229

WHY: Fetches raw PDF binary blob and creates a temporary browser URL for inline <iframe> preview.

WHEN: Active document changes in the reader.

HOW: apiClient.get(url, { responseType: 'blob' }) → URL.createObjectURL(response.data).

SECTION 09 // SYSTEM ARCHITECTURE



Reusable UI components: Protected routes, navigation shell, headers, cards, and RAG citations.

frontend/src/components/ProtectedRoute/ProtectedRoute.jsx

31 lines

ROUTE GUARD COMPONENT (JSX) • COMPONENT · GUARD

❓ WHY IS IT HERE?

Prevents visitors without active sessions from accessing private forum pages.

⏱️ WHEN DOES IT RUN?

Evaluated whenever a user tries to open a protected URL.

⚙️ HOW DOES IT WORK?

Reads useAuth(). If loading, renders neutral spinner. If !isAuthenticated, renders <Navigate to="/auth" state={{ from: location }} replace />.

💡 Beginner Concept Spotlight

Preserving Redirect State: By passing state={{ from: location }} in <Navigate>, after the user signs in, the login page knows where they originally wanted to go and redirects them straight back!

⚠️ Junior Developer Trap / Common Gotcha

Never redirect immediately if loading is true! When refreshing the page, AuthContext takes a millisecond to read localStorage. If you don't check loading, authenticated users will be kicked to login on every page refresh!

🎯 Technical Interview Pro-Tip

Q: Why return children directly if authenticated?
A: High-order component / wrapper pattern. ProtectedRoute acts as a clean boundary around any child element without modifying the child's internal props.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

ProtectedRoute Guard Logic

Lines 9-30

WHY:
Checks auth state and either displays loader, redirects to /auth, or renders children.

WHEN:
Mounts on protected route transition.

HOW:
if (loading) return spinner; if (!isAuthenticated) return <Navigate to="/auth" />; return children;

frontend/src/components/ProtectedRoute/ProtectedRoute.module.css

32 lines

PROTECTED ROUTE STYLES (CSS MODULE) • COMPONENT · CSS

❓ WHY IS IT HERE?

Styles the full-screen neutral session restore loader with smooth CSS rotation.

⏱️ WHEN DOES IT RUN?

Active during session verification on page load.

⚙️ HOW DOES IT WORK?

Uses flexbox centering and @keyframes protectedRouteSpin (rotate 360deg).

💡 Beginner Concept Spotlight

CSS Keyframe Animation: @keyframes defines intermediate steps in an animation sequence (e.g. from 0deg to 360deg). animation: ... linear infinite plays it continuously.

⚠️ Junior Developer Trap / Common Gotcha

Always set aria-hidden on purely decorative spinner divs to keep screen readers focused on accessible status text.

🎯 Technical Interview Pro-Tip

Q: Why use rem units for spinner dimensions instead of pixels?
A: Accessibility! If a user changes their browser root font size for vision assistance, rem units scale proportionally with user preferences.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Spinner & Animation Styles

Lines 6-31

WHY:
Centers loader and animates circular border indicator.

WHEN:
Displayed during auth loading state.

HOW:
border: 3px solid; border-top-color: var(--primary); animation: protectedRouteSpin 0.85s linear infinite.

frontend/src/components/Layout/Layout.jsx

90 lines

MAIN APP SHELL LAYOUT (JSX) • COMPONENT · SHELL

❓ WHY IS IT HERE?

Maintains consistent application chrome across all authenticated screens.

⏱️ WHEN DOES IT RUN?

Renders on all protected routes.

⚙️ HOW DOES IT WORK?

Renders fixed <Sidebar />, sticky <Navbar />, <Outlet /> inside <main>, and <footer />.

💡 Beginner Concept Spotlight

The <Outlet /> Component: In React Router, <Outlet /> is a placeholder that tells the router where to render child route components (Dashboard, PostQuestion, etc.) within a parent Layout shell.

⚠️ Junior Developer Trap / Common Gotcha

Keep getTitle() and getSubtitle() synchronized with your routes in App.jsx so page titles never lag behind the current URL.

🎯 Technical Interview Pro-Tip

Q: How does Layout.jsx handle responsive mobile screens?
A: Through CSS layout classes. On desktop, .layout__content has a margin-left: 17rem matching the fixed sidebar. On mobile, the layout collapses into a single column.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Dynamic Title & Subtitle Synchronization

Lines 16-40

WHY:

Syncs header titles with current location.pathname for instant user orientation.

WHEN:

Recalculates whenever route changes.

HOW:

Switches on location.pathname: /dashboard → "Home", /questions/ask → "Ask a question", etc.

App Chrome Structure

Lines 42-89

WHY:
Assembles Sidebar, Navbar, scrollable main container with <Outlet />, and footer.

WHEN:
Rendered on all authenticated views.

HOW:
<Sidebar /> + <Navbar /> + <main><Outlet /></main> + <footer />.

frontend/src/components/Layout/Layout.module.css

121 lines

LAYOUT SHELL STYLES (CSS MODULE) • COMPONENT · CSS

❓ WHY IS IT HERE?

Governs sidebar offset, scrollable content area, maximum widths, and footer positioning.

⏱️ WHEN DOES IT RUN?

Active on all protected pages.

⚙️ HOW DOES IT WORK?

Sets margin-left: var(--app-sidebar-width), min-height: 100vh, and flex layout.

💡 Beginner Concept Spotlight

Viewport Height (100vh): min-height: 100vh guarantees the layout always stretches to fill at least the full height of the user's screen, preventing the footer from floating in the middle of short pages.

⚠️ Junior Developer Trap / Common Gotcha

Always set max-width: var(--app-content-max, 80rem) with margin: 0 auto so content does not stretch awkwardly across ultra-wide monitors.

🎯 Technical Interview Pro-Tip

Q: What is the purpose of flex: 1 on .layout__main?
A: In a flex-direction: column container, flex: 1 forces the main content area to expand and occupy all remaining vertical space, pushing the footer neatly to the bottom of the viewport.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Shell Architecture & Main Column Width Constraints

Lines 5-34

WHY:

Offsets main content by sidebar width and centers content at 80rem max-width.

WHEN:

Applied to layout root and main content area.

HOW:

.layout__content { margin-left: 17rem; } .layout__mainInner { max-width: 80rem; margin: 0 auto; }

Footer Layout & Subtle Top Gradient

Lines 36-60

WHY:
Adds subtle top border and gradient separation for the educational footer.

WHEN:
Positioned at the bottom of the page.

HOW:
border-top: 1px solid var(--border); background-image: linear-gradient(...).

frontend/src/components/Navbar/Navbar.jsx

128 lines

NAVIGATION HEADER BAR (JSX) • COMPONENT · NAVBAR

❓ WHY IS IT HERE?

The primary top bar. Hosts real-time search with keyword debouncing, AI semantic search shortcut, user avatar, and logout.

⏱️ WHEN DOES IT RUN?

Rendered at the top of every authenticated page.

⚙️ HOW DOES IT WORK?

Uses useState, useEffect with 500ms setTimeout debounce, Lucide icons, and avatar image fallbacks.

💡 Beginner Concept Spotlight

Search Debouncing: If a user types "JavaScript", without debouncing, the app would send 10 API requests (J, Ja, Jav, Java...). Debouncing waits until the user stops typing for 500ms before sending a single clean request, reducing server load by 90%!

⚠️ Junior Developer Trap / Common Gotcha

Always return () ⇒ clearTimeout(delayDebounceFn) in your useEffect cleanup function, otherwise pending timers will execute and cause stale searches!

🎯 Technical Interview Pro-Tip

Q: Why synchronize search input state with URL search params (?q=...)?
A: Deep linking and shareability! If a user bookmarks or shares http://localhost:5173/dashboard?q=react, the Navbar reads the URL parameter on mount, populates the input, and restores the search state.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Search Term State & URL Sync

Lines 14-28

WHY:
Initializes input from URL params (?q or ?semantic) and stays synced.

WHEN:
Mounts or when URL parameters change.

HOW:
new URLSearchParams(location.search).get('q').

Debounced Keyword Search Effect (500ms Quiet Period)

Lines 31-44

WHY: Automatically navigates to /dashboard?q=<term> after user stops typing for 500ms.

WHEN: Runs on every keystroke in search input.

HOW: setTimeout(() ⇒ navigate(...), 500); return () ⇒ clearTimeout(...).

Semantic Search Action

Lines 46-58

WHY: Triggers AI semantic search mode (?semantic=...).

WHEN: User clicks "AI Search" button or presses enter.

HOW: navigate(`/dashboard?semantic=${encodeURIComponent(searchTerm)}`).

Navbar JSX Rendering

Lines 60-127

WHY: Renders title, search form with sparkles icon, user name, avatar, and logout button.

WHEN: Always visible at top of viewport.

HOW: Semantic HTML <header>, <form>, and accessible button controls.

frontend/src/components/Navbar/Navbar.module.css

212 lines

NAVBAR STYLES (CSS MODULE) • COMPONENT · CSS

❓ WHY IS IT HERE?

Styles the sticky navigation bar with glassmorphic border, search input padding, and badge colors.

⏱️ WHEN DOES IT RUN?

Active on all layout views.

⚙️ HOW DOES IT WORK?

position: sticky, top: 0, z-index: 40, flexbox alignment.

💡 Beginner Concept Spotlight

position: sticky vs position: fixed: fixed removes an element completely from the document flow and locks it to the viewport. sticky behaves like normal relative positioning until you scroll past it, at which point it sticks to the top without covering other elements.

⚠️ Junior Developer Trap / Common Gotcha

Always set a high z-index (e.g. z-index: 40) on sticky headers so scrolling cards and modals never slide over the top of the navigation bar.

🎯 Technical Interview Pro-Tip

Q: How does pointer-events: none improve search bar UX on the search icon?
A: It allows user mouse clicks on the magnifying glass icon to pass directly through to the underlying <input> element, focusing the input automatically.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Sticky Navbar Container

Lines 5-19

WHY:
Locks header to the top of the viewport during scrolling.

WHEN:
Active during user scrolling.

HOW:
position: sticky; top: 0; z-index: 40; border-bottom: 1px solid var(--border).

Search Input Styling

Lines 50-81

WHY:
Styles the search bar with left icon padding and right semantic button clearance.

WHEN:
Rendered in navbar.

HOW:
padding-left: 2.5rem; padding-right: 7rem; border-radius: var(--radius-lg).

AI Semantic Search Badge Button

Lines 83-123

WHY:
Highlights the AI search capability with orange branding inside the input box.

WHEN:
Appears when searchTerm length >= 3.

HOW:
position: absolute; right: 0.25rem; top: 50%; transform: translateY(-50%).

frontend/src/components/Sidebar/Sidebar.jsx

125 lines

APP NAVIGATION SIDEBAR (JSX) • COMPONENT · SIDEBAR

❓ WHY IS IT HERE?

The primary vertical navigation spine. Provides quick access to all core sections, displays the brand logo, user profile card, and logout action.

⏱️ WHEN DOES IT RUN?

Always visible on the left side of desktop screens.

⚙️ HOW DOES IT WORK?

Uses NavLink from react-router-dom with active class matching, Lucide icons, and useAuth().

💡 Beginner Concept Spotlight

NavLink vs. Link: <Link> only navigates. <NavLink> knows whether its target URL is currently active and provides an isActive boolean callback so you can apply active highlight styles automatically!

⚠️ Junior Developer Trap / Common Gotcha

Ensure brand logo click handlers include accessible keyboard handlers (onKeyDown for Enter and Space) so keyboard-only and screen reader users can navigate.

🎯 Technical Interview Pro-Tip

Q: Why is NavLink preferred over regular <a> tags in a React application?
A: Standard <a> tags cause the browser to trigger a full page reload, wiping out in-memory state. NavLink intercepts clicks, pushes to browser history, and triggers client-side route transitions without reloading.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Navigation Items Array

Lines 10-14

WHY:
Data-driven navigation definition. Adding a new route is as easy as adding one object here.

WHEN:
Evaluated when mapping sidebar links.

HOW:
Array of { icon, label, path } objects.

NavLink Mapping with Active Rail State

Lines 50-77

WHY:
Renders navigation links and dynamically applies active orange styling when URL matches.

WHEN:
Renders navigation list.

HOW:
className={({ isActive }) => `${styles.sidebar__link} ${isActive ? styles['sidebar__link--active'] : ''}`}

Sidebar Footer with New Question Button & User Card

Lines 80-120

WHY:
Provides primary "New Question" CTA and user avatar with role badge.

WHEN:
Anchored at the bottom of the sidebar.

HOW:
margin-top: auto pushes footer to the bottom of the flex container.

frontend/src/components/Sidebar/Sidebar.module.css

346 lines

SIDEBAR STYLES (CSS MODULE) • COMPONENT · CSS

❓ WHY IS IT HERE?

Defines the fixed 17rem left navigation column with BEM naming conventions.

⏱️ WHEN DOES IT RUN?

Styles the sidebar across all layout views.

⚙️ HOW DOES IT WORK?

position: fixed, left: 0, top: 0, height: 100vh, z-index: 50.

💡 Beginner Concept Spotlight

BEM (Block Element Modifier) Convention: CSS naming methodology: .sidebar (Block), .sidebar__link (Element), .sidebar__link--active (Modifier). Makes styles modular and prevents selector conflicts.

⚠️ Junior Developer Trap / Common Gotcha

Active indicators made with ::before pseudo-elements must have content: '' declared, otherwise browsers will not render the pseudo-element at all.

🎯 Technical Interview Pro-Tip

Q: How does margin-top: auto push elements to the bottom in flexbox?
A: In CSS flexbox, an auto margin absorbs all available free space in that direction. Setting margin-top: auto on .sidebar__footer absorbs all space between the nav links and the footer, pinning it to the bottom.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Fixed Sidebar Rail Container

Lines 4-19

WHY:

Locks sidebar to the left edge of the viewport.

WHEN:

Always visible on desktop.

HOW:

width: var(--app-sidebar-width, 15rem); position: fixed; left: 0; top: 0; height: 100vh.

Active Link Rail Accent (::before)

Lines 147-164

WHY:

Renders an elegant 3px vertical orange bar beside the currently active route.

WHEN:

Active on current route.

HOW:

.sidebar__link--active::before { position: absolute; left: 0.25rem; width: 3px; background-color: var(--primary); }

frontend/src/components/QuestionCard/QuestionCard.jsx

77 lines

QUESTION CARD ITEM (JSX) • COMPONENT · CARD

❓ WHY IS IT HERE?

The core feed item. Displays question title, author initials, cleaned markdown snippet, reply count, and "YOURS" badge.

⏱️ WHEN DOES IT RUN?

Rendered in lists on Dashboard and MyQuestions pages.

⚙️ HOW DOES IT WORK?

Uses cleanSnippet() regex to strip raw markdown formatting for previews, and links to /questions/:hash.

💡 Beginner Concept Spotlight

Stripping Markdown for Previews: Raw markdown contains hashtags (#), backticks (```), and links ([text](url)). Rendering raw markdown in a one-line preview looks ugly and broken. cleanSnippet() strips markup characters with regex.

⚠️ Junior Developer Trap / Common Gotcha

Always provide a unique key prop (key={q.questionHash || q.id}) when mapping lists in React, otherwise React cannot efficiently track item re-ordering.

🎯 Technical Interview Pro-Tip

Q: Why use an <article> HTML element for QuestionCard instead of a generic ?
A: Semantic HTML! <article> indicates a self-contained, independently distributable piece of content, which significantly improves SEO and accessibility for screen reader assistive technology.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

cleanSnippet Markdown Stripper

Lines 9-20

WHY:
Removes markdown fences, code blocks, bold/italics, and trims text to 160 characters with ellipsis.

WHEN:
Runs when preparing question content preview.

HOW:
Chained string replaces with regex: replace(/#+\s/g, '').replace(/```[\s\S]*?```/g, ' [code] ').

QuestionCard Component Rendering

Lines 22-76

WHY:
Renders avatar pill, title link, YOURS badge, excerpt, reply count badge, and relative date.

WHEN:
Rendered in question list feeds.

HOW:
<article className={styles.card}>
<Link to={`/questions/${hash}`}>
... </article>

frontend/src/components/QuestionCard/QuestionCard.module.css

116 lines

QUESTION CARD STYLES (CSS MODULE) • COMPONENT · CSS

❓ WHY IS IT HERE?

Styles feed items with border transitions, subtle hover elevation, and left orange accent for authored posts.

⏱️ WHEN DOES IT RUN?

Applied to question cards.

⚙️ HOW DOES IT WORK?

transition: border-color, box-shadow, transform; .card--yours has border-left: 3px solid var(--primary).

💡 Beginner Concept Spotlight

Card Hover States: Adding subtle transitions (transform: translateY(-1px) and box-shadow) gives cards a tactile, clickable feel that invites user engagement.

⚠️ Junior Developer Trap / Common Gotcha

Remember word-break: break-word on excerpts to prevent long URLs or code strings from overflowing the card boundary.

🎯 Technical Interview Pro-Tip

Q: How does the .card--yours class enhance usability?
A: It gives users immediate visual feedback on their own posts in a crowded cohort feed by rendering a distinctive left border highlight without needing to read the author name.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Card Base & Yours Modifier

Lines 1-20

WHY:

Provides white card background, rounded corners, and left orange border on user's own questions.

WHEN:

Applied to every question card.

HOW:

.card { background-color: var(--surface); border-radius: var(--radius-lg); } .card--yours { border-left: 3px solid var(--primary); }

yoursBadge Styling

Lines 66-75

WHY:
Small high-contrast badge highlighting authorship.

WHEN:
Displayed when isOwner is true.

HOW:
background-color: var(--primary-light); color: var(--primary-dark); font-weight: 700.

frontend/src/components/RagAnswerBody/RagAnswerBody.jsx

126 lines

RAG ANSWER & CITATION DISPLAY (JSX) • COMPONENT · RAG UI

❓ WHY IS IT HERE?

Renders AI markdown responses beautifully with custom styling for paragraphs, lists, blockquotes, and code blocks.

⏱️ WHEN DOES IT RUN?

Rendered whenever AI answers or question details are displayed.

⚙️ HOW DOES IT WORK?

Uses react-markdown with custom component overrides, Lucide Copy/Check icons, and navigator.clipboard.

💡 Beginner Concept Spotlight

Component Overrides in ReactMarkdown: By default, react-markdown renders standard unstyled <pre> and <code> tags. Passing custom components={{ code: ..., pre: ... }} lets us replace them with rich interactive widgets featuring copy buttons!

⚠️ Junior Developer Trap / Common Gotcha

Always check if children is a non-empty string before rendering ReactMarkdown to avoid rendering blank containers.

🎯 Technical Interview Pro-Tip

Q: How does CodeBlock implement the copy-to-clipboard feedback loop?
A: navigator.clipboard.writeText(raw) copies code to clipboard; setCopied(true) temporarily swaps the copy icon with a green checkmark; setTimeout(() ⇒ setCopied(false), 2000) resets it back after 2 seconds.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

ReactMarkdown Custom Element Mapping

Lines 18-74

WHY:

Overrides HTML tags (p, ul, ol, h2, blockquote, a, code, pre) with CSS Module classes.

WHEN:

Renders markdown text.

HOW:

components={{ p: ({ node, ...props }) ⇒ <p className={styles.p} {...props} /> }}.

CodeBlock Component with Copy Button

Lines 76-125

WHY:
Displays code syntax with language label badge and one-click clipboard copy button.

WHEN:
Rendered on fenced code blocks (```js ... ```).

HOW:
useRef to grab raw code, navigator.clipboard.writeText, copied state toggle with 2-second timer.

frontend/src/components/RagAnswerBody/RagAnswerBody.module.css

157 lines

RAG ANSWER STYLES (CSS MODULE) • COMPONENT · CSS

❓ WHY IS IT HERE?

Provides sleek styling for markdown prose, code blocks, copy buttons, and callouts.

⏱️ WHEN DOES IT RUN?

Styles markdown outputs.

⚙️ HOW DOES IT WORK?

Dark code container with monospace fonts, orange blockquote borders, and subtle link underlines.

💡 Beginner Concept Spotlight

Monospace Typography: Code blocks must use monospace fonts (where every character has identical width, like ui-monospace, SFMono, Consolas) so indentation and code columns align perfectly.

⚠️ Junior Developer Trap / Common Gotcha

Make sure code blocks have overflow-x: auto so long lines of code can scroll horizontally without breaking the page layout.

🎯 Technical Interview Pro-Tip

Q: Why use CSS variables in code toolbar styling?
A: It maintains cohesive theme integration, ensuring borders, background contrasts, and typography match the rest of the application.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Blockquote Callout Styling

Lines 47-54

WHY:
Renders quoted text with a 3px primary orange accent line.

WHEN:
Rendered on markdown blockquotes (> Quote).

HOW:
border-left: 3px solid var(--primary); background-color: var(--surface-container-low).

codeWrap & codeToolbar Styles

Lines 83-127

WHY:
Gives code blocks an editor appearance with dark background, language pill, and copy button.

WHEN:
Wraps code snippets.

HOW:
background-color: var(--tertiary); overflow: hidden; .codeToolbar has rgba background.

SECTION 010 // SYSTEM ARCHITECTURE



All public and authenticated pages: Landing, Login/Register, Feed, Question Ask, Detail, and RAG Workspace.

frontend/src/pages/Landing/Landing.jsx

509 lines

PUBLIC MARKETING PAGE (JSX) • PAGE · LANDING

❓ WHY IS IT HERE?

The public front door of ForumStudy. Introduces learners to cohort study, explains RAG with animated diagrams, and routes users to sign up.

⏱️ WHEN DOES IT RUN?

Rendered when visiting the root path (/).

⚙️ HOW DOES IT WORK?

Uses Framer Motion for scroll/entry animations, Lucide icons, smooth scrolling helpers, and useAuth() state.

💡 Beginner Concept Spotlight

Framer Motion Animations: Motion.div accepts initial={{ opacity: 0, y: 12 }} and animate={{ opacity: 1, y: 0 }} to smoothly slide content into place when the page mounts.

⚠️ Junior Developer Trap / Common Gotcha

Notice how the CTA button changes intelligently: if isAuthenticted is true, it displays "Open forum" and navigates to /dashboard; if false, it displays "Sign in" and goes to /auth!

🎯 Technical Interview Pro-Tip

Q: How does Landing.jsx implement smooth scrolling to sections without external libraries?
A: document.getElementById('how-it-works')?.scrollIntoView({ behavior: 'smooth' }). Native browser API provides buttery smooth scrolling.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Smooth Scrolling Anchors

Lines 27-37

WHY:
Scrolls viewport smoothly to "How It Works" and "Course RAG" sections.

WHEN:
User clicks navigation buttons in the landing header.

HOW:
document.getElementById(...).scrollIntoView({ behavior: 'smooth' }).

Hero Section with Framer Motion

Lines 118-181

WHY:
Engaging title, subtitle, CTA buttons, and feature checklist panel.

WHEN:
Rendered in hero viewport.

HOW:
Motion.h1, Motion.p, Motion.div with staggered transition delays.

Course RAG Educational Pipeline Breakdown

Lines 217-285

WHY:
Explains the 3-step RAG concept (Ingest & chunk → Retrieve at question time → Grounded responses).

WHEN:
User scrolls down to learn how AI RAG works.

HOW:
Visual step cards with icons, headers, and conceptual explanations.

frontend/src/pages/Landing/Landing.module.css

680 lines

LANDING PAGE STYLES (CSS MODULE) • PAGE · CSS

❓ WHY IS IT HERE?

Provides responsive layout rules, sticky marketing header, card grids, and footer styling.

⏱️ WHEN DOES IT RUN?

Styles the Landing page.

⚙️ HOW DOES IT WORK?

CSS Grid with clamp() typography and responsive media queries.

💡 Beginner Concept Spotlight

Fluid Typography with clamp(): font-size: clamp(1.75rem, 4vw, 2.5rem) means the font scales smoothly with the screen width (4vw) but will never shrink below 1.75rem or grow larger than 2.5rem!

⚠️ Junior Developer Trap / Common Gotcha

Always test responsive grids on mobile screens. Media queries (min-width: 640px, min-width: 900px) progressively enhance columns from 1fr to 2fr to 4fr.

🎯 Technical Interview Pro-Tip

Q: What is the benefit of CSS clamp() over traditional media queries?
A: It eliminates jarring breakpoint jumps by smoothly interpolating font sizes across any display width.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Hero Responsive Grid

Lines 202-230

WHY:
Splits hero into text copy and side panel on desktops >= 960px.

WHEN:
Applied to hero section.

HOW:
grid-template-columns: 1fr minmax(16rem, 22rem); gap: var(--spacing-3xl).

Responsive Feature Cards Grid

Lines 441-457

WHY:

Single column on mobile, 2 columns on tablets, 4 columns on wide screens.

WHEN:

Applied to capabilities section.

HOW:

@media (min-width: 640px) { repeat(2, 1fr) } @media (min-width: 1100px) { repeat(4, 1fr) }

frontend/src/pages/Auth/Auth.jsx

380 lines

AUTHENTICATION PAGE (JSX) • PAGE · AUTH

❓ WHY IS IT HERE?

The unified login and registration portal. Seamlessly transitions between Sign In and Sign Up states without page reloads.

⏱️ WHEN DOES IT RUN?

Rendered at /auth.

⚙️ HOW DOES IT WORK?

AnimatePresence mode="wait", form state management, regex email validation, password visibility toggle, and useAuth() calls.

💡 Beginner Concept Spotlight

Single Route Mode Switching: Instead of having two separate URLs (/login and /register), keeping them on /auth with an isLogin state boolean makes the switch instantaneous and retains shared branding context.

⚠️ Junior Developer Trap / Common Gotcha

After successful registration, notice lines 119-124: The app automatically switches to the login form after a 1.5-second success message, providing a delightful user experience.

🎯 Technical Interview Pro-Tip

Q: How does password visibility toggling work in React?
A: An isShowPassword boolean state dynamically toggles the input's HTML attribute: type={showPassword ? "text" : "password"}.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Client-Side Form Validation Guard

Lines 38-85

WHY:
Validates email regex, password minimum length (6), and name lengths before making network requests.

WHEN:
User clicks Submit button.

HOW:
Checks emailPattern.test(email), password.length >= 6, and sets error state if invalid.

Asynchronous Submit Handler

Lines 89-130

WHY:
Dispatches login() or register() via AuthContext and handles success/error messages.

WHEN:
Form submission on valid inputs.

HOW:
await login({ email, password }) or await register({ firstName, lastName, email, password }).

AnimatePresence Animated Form Swap

Lines 225-232

WHY:
Smoothly fades and slides form elements when toggling between login and signup.

WHEN:
User clicks "Create an account" or "Back to sign in".

HOW:
Framer Motion AnimatePresence mode="wait" with opacity and y transitions.

frontend/src/pages/Auth/Auth.module.css

538 lines

AUTH PAGE STYLES (CSS MODULE) • PAGE · CSS

❓ WHY IS IT HERE?

Splits the authentication screen into a branded gradient hero panel (45%) and a card form section (55%).

⏱️ WHEN DOES IT RUN?

Styles the /auth page.

⚙️ HOW DOES IT WORK?

linear-gradient(155deg, ...), backdrop-filter: blur(10px), rounded input boxes, and focus ring transitions.

💡 Beginner Concept Spotlight

backdrop-filter: blur(): Creates modern "glassmorphism" by blurring whatever background elements sit behind a translucent container.

⚠️ Junior Developer Trap / Common Gotcha

Ensure inputs have type="password" by default so screen shoulder-surfers cannot view sensitive passwords.

🎯 Technical Interview Pro-Tip

Q: What is the benefit of a split-screen authentication layout?
A: The left panel reinforces brand trust and value propositions while the right panel keeps the form focused and uncluttered.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Left Branded Info Panel

Lines 18-37

WHY: Displays orange gradient with value pillars and cohort social proof.

WHEN: Desktop screens >= 768px.

HOW: background: linear-gradient(155deg, var(--primary), var(--primary-dark)); width: 45%.

Right Form Container Card

Lines 236-252

WHY:
Clean elevated card holding the login/registration input fields.

WHEN:
Always visible in form area.

HOW:
max-width: 28rem; background-color: var(--surface); border-radius: var(--radius-xl).

frontend/src/pages/Dashboard/Dashboard.jsx

306 lines

MAIN DASHBOARD & QUESTION FEED (JSX) • PAGE · DASHBOARD

❓ WHY IS IT HERE?

The primary activity hub. Displays personalized greetings, quick action tiles, live community metrics, search filter pills, and question cards.

⏱️ WHEN DOES IT RUN?

Rendered at /dashboard.

⚙️ HOW DOES IT WORK?

Reads searchParams via useSearchParams(), calculates stats via useMemo(), renders skeleton loaders while fetching, and lists questions.

💡 Beginner Concept Spotlight

useMemo() for Performance: Calculating stats (total replies, unanswered, authored by me) over 100 questions takes CPU cycles. useMemo() caches the calculated result and only recomputes if the questions array actually changes!

⚠️ Junior Developer Trap / Common Gotcha

Skeleton loaders provide a far superior user experience compared to a jarring empty white screen or tiny spinner. They mimic the layout of incoming cards.

🎯 Technical Interview Pro-Tip

Q: How does Dashboard.jsx determine whether to run semantic AI search or keyword search?
A: It checks URL parameters: if semanticQuery exists in searchParams, it calls questionService.searchQuestionsSemantic(); if keywordQuery exists, it calls questionService.getQuestions({ search }).

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

fetchQuestions Data Loader

Lines 33-57

WHY:

Fetches question list based on active search mode (semantic vs keyword vs default feed).

WHEN:

Mounts or when searchParams change.

HOW:

useCallback with searchQuestionsSemantic or getQuestions, setting questions state.

useMemo Stats Calculator

Lines 64-83

WHY: Aggregates total questions, total replies, unanswered count, and authored-by-me count.

WHEN: Recomputes when questions or user changes.

HOW: Iterates questions array, accumulating answerCounts and comparing authorship.

Stats Grid Row

Lines 155-178

WHY: Displays 4 statistical metric tiles keeping learners informed of community activity.

WHEN: Rendered at top of feed.

HOW: Grid of stat cards: Questions, Replies, Unanswered, Yours.

Animated Skeleton Card Loaders

Lines 220-233

WHY: Prevents layout shift by showing gray placeholder pulse cards while data loads.

WHEN: Displayed when loading is true.

HOW: Maps 4 dummy items rendering .skeletonCard with CSS pulse animation.

frontend/src/pages/Dashboard/Dashboard.module.css

437 lines

DASHBOARD PAGE STYLES (CSS MODULE) • PAGE · CSS

❓ WHY IS IT HERE?

Governs the visual hierarchy of the main feed: hero cards, action tiles, stats metrics, and skeleton loader pulse animations.

⏱️ WHEN DOES IT RUN?

Styles the /dashboard view.

⚙️ HOW DOES IT WORK?

Grid templates, subtle border transitions, and @keyframes pulse.

💡 Beginner Concept Spotlight

CSS Skeleton Animation: Animating opacity from 0.6 to 1.0 continuously creates a smooth "breathing" effect that signals to the user that content is actively loading.

⚠️ Junior Developer Trap / Common Gotcha

Always set overflow: hidden and text-overflow: ellipsis on titles in grid cards so unusually long question titles do not break grid alignment.

🎯 Technical Interview Pro-Tip

Q: Why use responsive CSS grid with repeat(auto-fit, minmax(...))?
A: It creates automatically wrapping card columns that dynamically adapt to any screen size without needing dozens of hardcoded media query breakpoints.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Quick Action Tiles Grid

Lines 48-78

WHY:
Interactive shortcut cards for "New question", "Your topics", and "Knowledge base".

WHEN:
Rendered in hero section.

HOW:
grid-template-columns: repeat(3, 1fr); hover elevates card with translateY(-1px).

Skeleton Pulse Animation

Lines 296-306

WHY:
Gives loading cards a polished shimmer effect.

WHEN:
Active during data fetching.

HOW:
@keyframes pulse { 0% { opacity: 0.6 } 50% { opacity: 1 } 100% { opacity: 0.6 } }

frontend/src/pages/PostQuestion/PostQuestion.jsx

403 lines

ASK QUESTION PAGE WITH AI COACH (JSX) • PAGE · POST QUESTION

❓ WHY IS IT HERE?

Enables learners to ask well-structured technical questions. Includes a helpful formatting toolbar and an AI Draft Coach that reviews questions before posting.

⏱️ WHEN DOES IT RUN?

Rendered at /questions/ask.

⚙️ HOW DOES IT WORK?

Manages title and content state, inserts markdown formatting via textarea selection ranges, calls AI coach service, and publishes to backend.

💡 Beginner Concept Spotlight

Selection Range Manipulation: textarea.selectionStart and selectionEnd detect which text the user highlighted with their mouse. We can wrap only that selected word in markdown asterisks (**bold**) without losing the user's typing cursor!

⚠️ Junior Developer Trap / Common Gotcha

Always disable the submit and coach buttons while requests are in flight (disabled={coachLoading || submitting}) to prevent accidental double-posting.

🎯 Technical Interview Pro-Tip

Q: How does the AI Draft Coach help students in a learning community?
A: Students often post vague questions like "my code doesn't work". The AI Draft Coach analyzes their draft in 1 second and provides actionable checklist tips: "Add the exact error message", "State what you expected", "Include code snippet".

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

insertMarkdown Textarea Helper

Lines 50-79

WHY:
Inserts markdown formatting (**bold**, *italic*, ```code```, [link]) around user-selected text.

WHEN:
User clicks toolbar buttons.

HOW:
Uses textareaRef.current, slices text, inserts formatting, and restores cursor focus.

handleGetCoachFeedback Function

Lines 82-107

WHY: Sends current draft to Gemini Flash and displays 3-5 constructive tips in a dismissible green panel.

WHEN: User clicks "AI suggestions".

HOW: questionService.generateQuestionDraftCoach({ title, content }) → sets coachTips array.

handleSubmit Publisher

Lines 110-140

WHY: Validates inputs, publishes question to backend, and redirects to the new thread view.

WHEN: User clicks "Post Question".

HOW: questionService.createQuestion({ title, content }) → navigate(`/questions/${hash}`).

Top Guidance & Quality Checklist Callout

Lines 158-198

WHY: Reminds learners of forum posting etiquette (runnable context, repro steps, minimal code).

WHEN: Permanently displayed above form.

HOW: Callout box with clear checklist items.

frontend/src/pages/PostQuestion/PostQuestion.module.css

417 lines

POST QUESTION STYLES (CSS MODULE) • PAGE · CSS

❓ WHY IS IT HERE?

Styles the guidance card, markdown editor toolbar, AI feedback alert box, and action buttons.

⏱️ WHEN DOES IT RUN?

Styles the /questions/ask page.

⚙️ HOW DOES IT WORK?

Editor wrapper with focus-within highlighting, green coach panel, and spinning loader icon.

💡 Beginner Concept Spotlight

:focus-within Pseudo-Class: When a user clicks inside the <textarea>, :focus-within on the outer wrapper (.editorWrap:focus-within) highlights the entire border and toolbar in orange!

⚠️ Junior Developer Trap / Common Gotcha

Always provide resize: vertical on textareas so users can expand the writing area vertically without breaking horizontal page boundaries.

🎯 Technical Interview Pro-Tip

Q: What is the benefit of the :focus-within pseudo-selector?
A: It allows parent containers to style themselves based on child focus without needing complex JavaScript focus/blur event listeners.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Guidance Callout Box Styles

Lines 48-58

WHY:
Highlights best practices with a 4px primary orange left accent.

WHEN:
Rendered at top of form.

HOW:
border-left: 4px solid var(--primary); background-color: var(--surface-container-low).

Editor Wrap & :focus-within

Lines 157-168

WHY:
Creates an integrated container uniting the toolbar and textarea.

WHEN:
User focuses textarea.

HOW:
.editorWrap:focus-within { border-color: var(--primary); box-shadow: 0 0 0 3px rgba(249, 115, 22, 0.15); }

AI Coach Feedback Panel Styles

Lines 279-317

WHY:
Renders an attractive emerald-tinted suggestion box with sparkler icon.

WHEN:
Displayed when coachTips is populated.

HOW:
background-color: #f0fdf4; border: 1px solid #bbf7d0; color: #166534.

frontend/src/pages/QuestionDetail/QuestionDetail.jsx

605 lines

QUESTION DETAIL & ANSWER WORKSPACE (JSX) • PAGE · QUESTION DETAIL

❓ WHY IS IT HERE?

The core discussion thread. Displays the full question, rendered markdown, community answers, related questions sidebar, and AI Answer-Fit relevance checker.

⏱️ WHEN DOES IT RUN?

Rendered when opening any question (/questions/:questionHash).

⚙️ HOW DOES IT WORK?

Fetches question details and parallel related questions, renders answers, manages answer composition form, and posts replies.

💡 Beginner Concept Spotlight

Parallel Asynchronous Data Fetching: Notice lines 65-75: Once the question is loaded, the page immediately launches a parallel request to fetch related questions (questionService.findSimilarQuestions), keeping the UI snappy!

⚠️ Junior Developer Trap / Common Gotcha

Notice lines 407-418: If the logged-in user authored this question, the answer form is automatically hidden and replaced with "You cannot answer your own question", reinforcing the backend business rule.

🎯 Technical Interview Pro-Tip

Q: How does the AI Answer-Fit feature help maintain answer quality?
A: Learners can click "Check draft fit" before posting. Gemini evaluates whether their draft genuinely addresses the question or goes off-topic, giving a "Strong match", "Partial match", or "Weak match" rating.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

fetchQuestionDetails & Parallel Similar Questions

Lines 55-90

WHY:
Loads thread details and queries similar questions using the question title.

WHEN:
Mounts or when URL questionHash changes.

HOW:
Awaits getSingleQuestion(activeHash); runs findSimilarQuestions in parallel.

handleCheckFit AI Relevance Evaluator

Lines 137-159

WHY: Sends draft answer to Gemini to evaluate relevance against the question.

WHEN: User clicks "Check draft fit".

HOW: questionService.assessAnswerFit(activeHash, answerContent) → sets fitResult { level, note }.

handleSubmitAnswer Reply Publisher

Lines 162-197

WHY: Submits community answer to backend and appends it to the answers list.

WHEN: User clicks "Post Your Answer".

HOW: answerService.postAnswer({ questionId, content }) → updates answers state array.

Community Answers Section

Lines 343-403

WHY: Renders list of community replies with author initials, dates, and formatted markdown.

WHEN: Rendered beneath question body.

HOW: Maps answers array, rendering <RagAnswerBody> {answer.content} </RagAnswerBody>.

Related Questions Sidebar

Lines 564-601

WHY: Displays similar forum topics recommended by AI vector cosine similarity.

WHEN: Rendered in right column.

HOW: Maps relatedQuestions array with links to /questions/:hash.

frontend/src/pages/QuestionDetail/QuestionDetail.module.css

663 lines

QUESTION DETAIL STYLES (CSS MODULE) • PAGE · CSS

❓ WHY IS IT HERE?

Governs the dual-column discussion layout: 1fr main content column and 20rem related questions sidebar.

⏱️ WHEN DOES IT RUN?

Styles the /questions/:questionHash view.

⚙️ HOW DOES IT WORK?

grid-template-columns: 1fr 20rem, responsive media queries collapsing to 1fr on screens <= 960px.

💡 Beginner Concept Spotlight

CSS Grid with Fixed Sidebar (1fr 20rem): The main column takes all available flexible space (1fr), while the sidebar stays fixed at 20rem wide. On mobile screens (<= 960px), it collapses into a single column (1fr).

⚠️ Junior Developer Trap / Common Gotcha

Always set min-width: 0 on flexible grid columns (mainColumn), otherwise wide code blocks can force the grid column to expand beyond the viewport!

🎯 Technical Interview Pro-Tip

Q: Why is min-width: 0 necessary on flex/grid children containing code blocks?
A: By default, flex/grid items have min-width: auto. If an item contains pre-formatted text or code that is wider than the column, it will refuse to shrink below its content size, causing horizontal overflow. min-width: 0 overrides this behavior.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Two-Column Grid Layout

Lines 31-43

WHY:

Main discussion thread on the left, related questions on the right.

WHEN:

Desktop displays > 960px.

HOW:

grid-template-columns: 1fr 20rem; @media (max-width: 960px) { grid-template-columns: 1fr; }

AI Fit Result Badges

Lines 413-437

WHY:
Color-coded feedback: Strong match (emerald), Partial match (amber), Weak match (red).

WHEN:
Rendered after running AI fit check.

HOW:
.fitResultBox--strong, .fitResultBox--partial, .fitResultBox--weak.

frontend/src/pages/MyQuestions/MyQuestions.jsx

133 lines

MY QUESTIONS PAGE (JSX) • PAGE · MY QUESTIONS

❓ WHY IS IT HERE?

A personal workspace where learners can track their own questions, monitor incoming answers, and view thread activity.

⏱️ WHEN DOES IT RUN?

Rendered at /my-questions.

⚙️ HOW DOES IT WORK?

Calls questionService.getQuestions({ mine: true }) and renders QuestionCard components.

💡 Beginner Concept Spotlight

Reusing Components with Props: Notice how MyQuestions uses the exact same <QuestionCard /> component as the Dashboard feed. Reusing components saves hundreds of lines of duplicate code!

⚠️ Junior Developer Trap / Common Gotcha

Always provide a dedicated empty state explaining that questions will appear here once posted, along with an "Ask your first question" button.

🎯 Technical Interview Pro-Tip

Q: How does the backend know which questions belong to "mine: true"?
A: The frontend sends the Bearer token in the header. The backend authentication middleware extracts req.user.id. When mine=true, the SQL query appends AND q.user_id = req.user.id.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

fetchMyQuestions Handler

Lines 17-29

WHY: Queries questions filtered for the current authenticated user.

WHEN: Mounts on page load.

HOW: questionService.getQuestions({ mine: true }) → setQuestions(res.questions).

Header Card with New Question Action

Lines 38-58

WHY:
Displays workspace title and quick button to ask a question.

WHEN:
Rendered at top of page.

HOW:
Header card with navigation button to /questions/ask.

Personal Empty State

Lines 95-116

WHY:
Encourages the user to post their first question if they haven't created any topics yet.

WHEN:
questions.length === 0.

HOW:
Card with message icon and CTA button.

frontend/src/pages/MyQuestions/MyQuestions.module.css

251 lines

MY QUESTIONS STYLES (CSS MODULE) • PAGE · CSS

❓ WHY IS IT HERE?

Styles the personalized topics feed, header card, skeleton loaders, and empty state.

⏱️ WHEN DOES IT RUN?

Styles the /my-questions view.

⚙️ HOW DOES IT WORK?

Card styling consistent with Dashboard design tokens.

💡 Beginner Concept Spotlight

Design System Consistency: By reusing the same border radii, padding, and color tokens as Dashboard.module.css, the application feels unified and professional.

⚠️ Junior Developer Trap / Common Gotcha

Ensure action buttons have cursor: pointer and clear hover transitions.

🎯 Technical Interview Pro-Tip

Q: What role do CSS variables play in maintaining consistency between Dashboard and MyQuestions?
A: Both modules reference var(--surface), var(--border), and var(--primary), guaranteeing identical visual weights across screens.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

headerCard Styles

Lines 8-18

WHY:

Creates the elevated top workspace banner.

WHEN:

Rendered at top of page.

HOW:

background-color: var(--surface);
border-radius: var(--radius-xl);
padding: 1.75rem 2rem.

frontend/src/pages/RagDocuments/RagDocuments.jsx

618 lines

RAG DOCUMENTS & PDF AI WORKSPACE (JSX) • PAGE · RAG DOCUMENTS

❓ WHY IS IT HERE?

The ultimate Document AI workspace. Allows students to build a private PDF library, preview documents inline, search chunks semantically, and ask questions answered strictly from PDF excerpts.

⏱️ WHEN DOES IT RUN?

Rendered at /rag-documents.

⚙️ HOW DOES IT WORK?

Multi-state React component handling file upload, polling status every 3s, PDF blob URLs, semantic chunk search, and grounded AI queries.

💡 Beginner Concept Spotlight

Split-Screen Workspace UI: Left column (22rem) holds the Document Library and Upload Dropzone. Right column displays the active PDF reader, semantic search form, and grounded AI question answering panel!

⚠️ Junior Developer Trap / Common Gotcha

Remember to poll document status (ragService.pollDocumentStatus) after upload because PDF text extraction and embedding generation run in the background!

🎯 Technical Interview Pro-Tip

Q: How does RagDocuments ensure answers are grounded in the user's uploaded document rather than hallucinated by AI?
A: The backend semantic search retrieves the top matching chunks from that specific document and passes them as the only permitted knowledge context to the Gemini prompt. The response citations list exact chunk indices.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Active Document Blob URL Loader

Lines 73-104

WHY:
Fetches raw PDF binary and creates an object URL for the inline <iframe> preview.

WHEN:
Runs whenever activeDoc changes.

HOW:
ragService.fetchPdfObjectUrl(doc.id) → setPdfUrl(url); revokes previous URL on cleanup.

handleUpload and Status Polling Loop

Lines 124-169

WHY: Uploads PDF, adds document to list immediately as "processing", and polls every 3 seconds until status is "ready".

WHEN: User selects PDF and clicks Upload.

HOW: ragService.uploadPdf(selectedFile) → ragService.pollDocumentStatus(doc.id, updateCallback).

handleSearchSubmit (In-Document Semantic Search)

Lines 191-209

WHY: Searches paragraphs in the active document matching a semantic concept.

WHEN: User submits search query.

HOW: ragService.searchInDocument(doc.id, query) → renders matching excerpts with percentage match badges.

handleAskSubmit (Grounded AI Q&A)

Lines 212-230

WHY: Asks Gemini a question answered strictly from the PDF with citations.

WHEN: User submits a question in the "Ask with AI" panel.

HOW: ragService.queryDocument(doc.id, aiQuestion) → renders grounded response with <RagAnswerBody>.

Section 1: Inline PDF Reader Preview

Lines 416-454

WHY: Renders the live PDF document inside an embedded <iframe>.

WHEN: Document is selected.

HOW: <iframe src={pdfUrl} className={styles.pdfIframe} />

Section 2: Semantic Search in Document

Lines 457-540

WHY: Form and ranked chunk excerpts cards with similarity percentage badges.

WHEN: User searches text.

HOW: Displays matching chunks with Chunk # and % match scores.

Section 3: Ask with AI Grounded Response Panel

Lines 543-611

WHY:
Textarea input for questions and green AI answer card with citations.

WHEN:
User asks question about document.

HOW:
Form submission → renders <RagAnswerBody>{aiAnswer}</RagAnswerBody>.

frontend/src/pages/RagDocuments/RagDocuments.module.css

629 lines

RAG WORKSPACE STYLES (CSS MODULE) • PAGE · CSS

❓ WHY IS IT HERE?

Governs the layout of the Document AI workspace: 22rem library sidebar and full-height interactive active document view.

⏱️ WHEN DOES IT RUN?

Styles the /rag-documents view.

⚙️ HOW DOES IT WORK?

grid-template-columns: 22rem 1fr; PDF iframe height: 24rem; citation badges.

💡 Beginner Concept Spotlight

Fixed-Width Sidebar with Fluid Canvas (22rem 1fr): The library sidebar stays at 22rem wide so long document titles fit comfortably, while the right canvas expands to fill the remainder of the screen for reading and AI interaction.

⚠️ Junior Developer Trap / Common Gotcha

Always set overflow: hidden on the iframe preview container with border-radius: var(--radius-md) to prevent the native browser PDF scrollbars from spilling outside the rounded corners.

🎯 Technical Interview Pro-Tip

Q: How does RagDocuments.module.css handle responsive mobile screens?
A: At @media (max-width: 1024px), the layout switches from a two-column grid to grid-template-columns: 1fr, stacking the library and the active reader vertically for tablets and phones.

🔍 Line-by-Line & Block-by-Block Anatomical Breakdown

Two-Column Workspace Grid

Lines 43-54

WHY: Splits library column (22rem) and active document workspace (1fr).

WHEN: Desktop displays > 1024px.

HOW: grid-template-columns: 22rem 1fr; gap: 1.75rem.

PDF Preview Container & Iframe

Lines 377-393

WHY:
Houses the embedded PDF viewer with rounded corners and clean borders.

WHEN:
Viewing active PDF.

HOW:
.previewContainer { width: 100%; height: 24rem; overflow: hidden; } .pdfIframe { width: 100%; height: 100%; border: none; }

Similarity Match Badges

Lines 521-530

WHY:
Renders sleek blue badges showing similarity score percentage (e.g. 88% match).

WHEN:
Rendered in semantic search excerpts.

HOW:
background-color: #dbeafe; color: #1e40af; border-radius: var(--radius-sm).

© 2026 Evangadi Forum Study Group. All rights reserved.